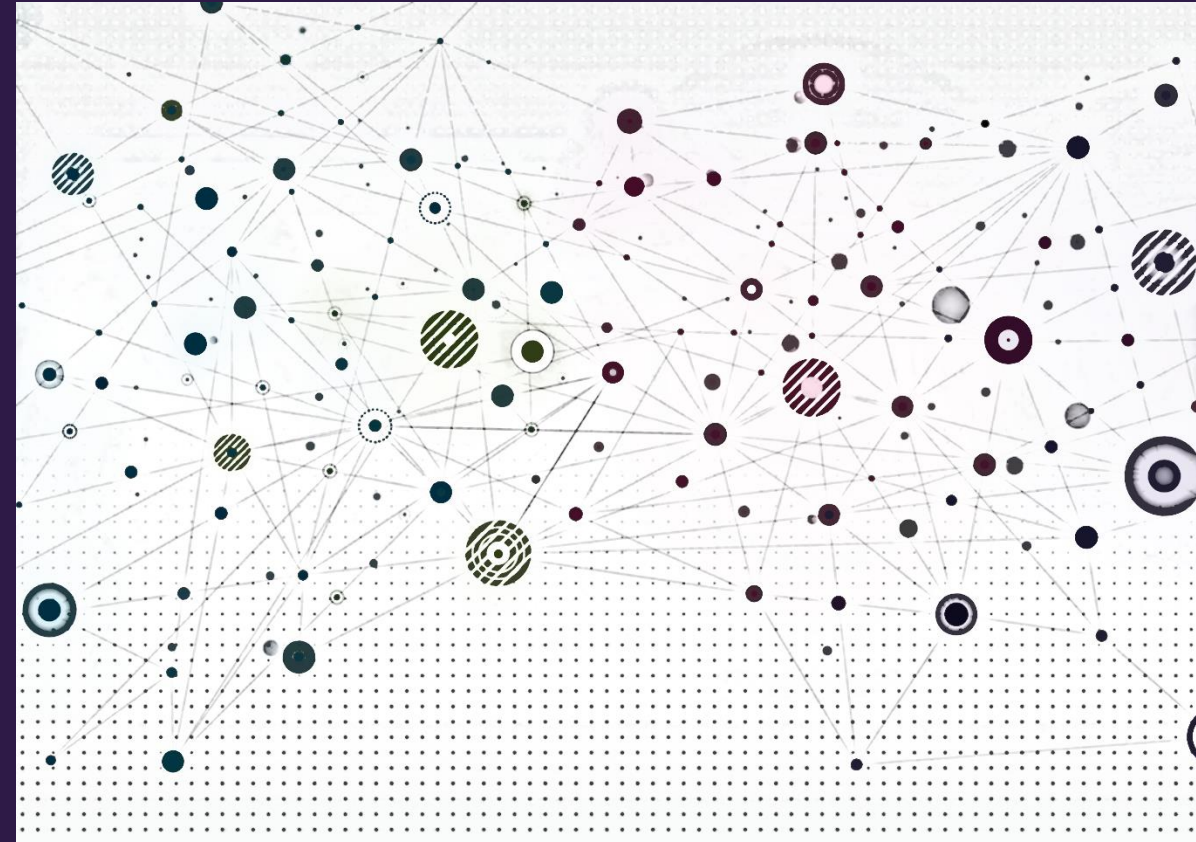


Introduction to Graph Neural Networks





Overview

- Recap
- Deep learning meets graphs
- Graph convolutional networks



Recap

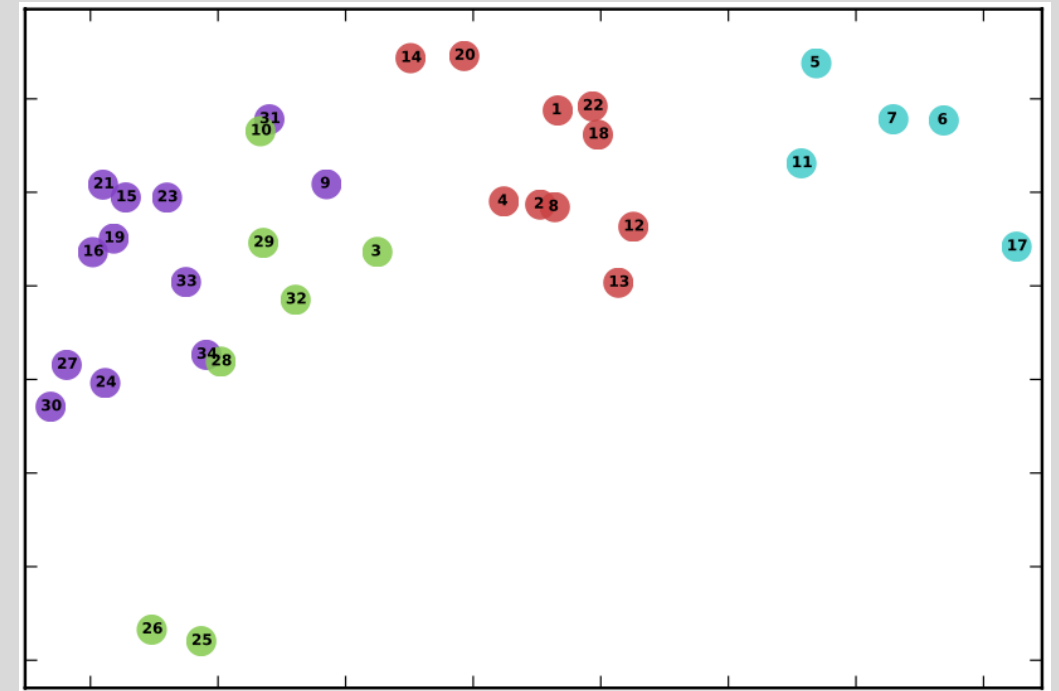
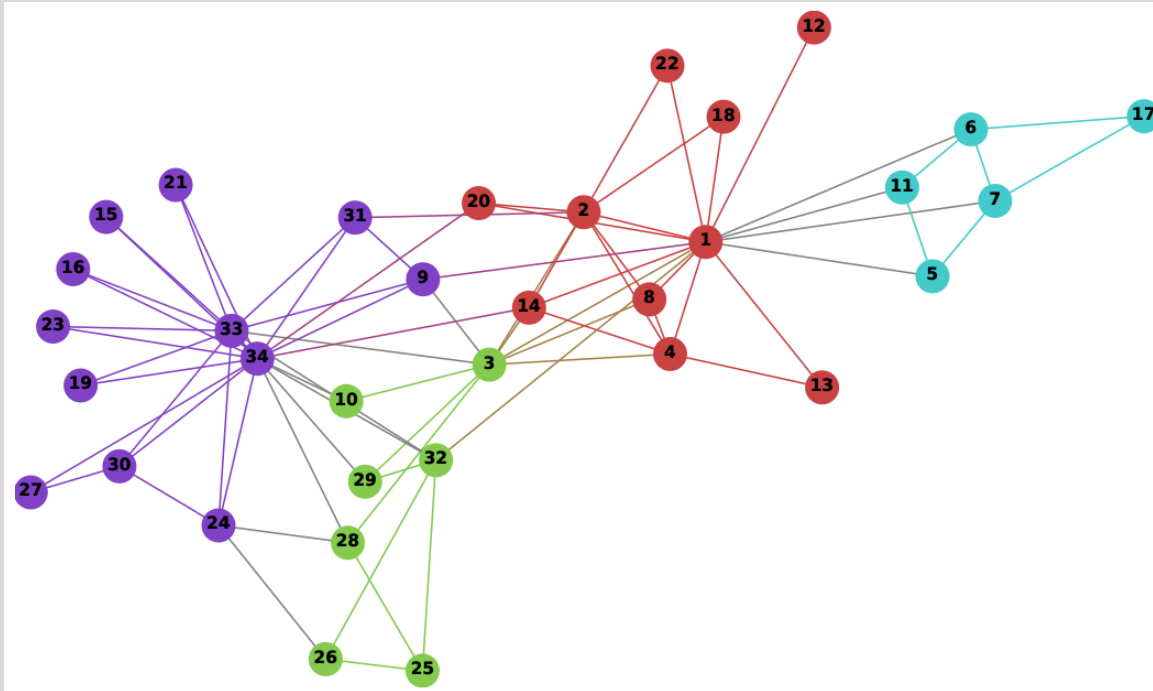




Node embeddings

Goal: Map nodes to $\mathbb{R}^d$ such that *similar* nodes are *close* to each other (e.g., have high cosine similarity)

Task: How to learn the mapping function?





Node embeddings

- Define an *encoder*, $ENC(v) = \mathbf{z}_v$, that maps v to a vector, $\mathbf{z}_v \in \mathbb{R}^d$
- Define a *decoder* $DEC(u, v) \approx \mathbf{S}[u, v]$ where $\mathbf{S}$ is a similarity function
- Define the encoder, decoder, and similarity and train

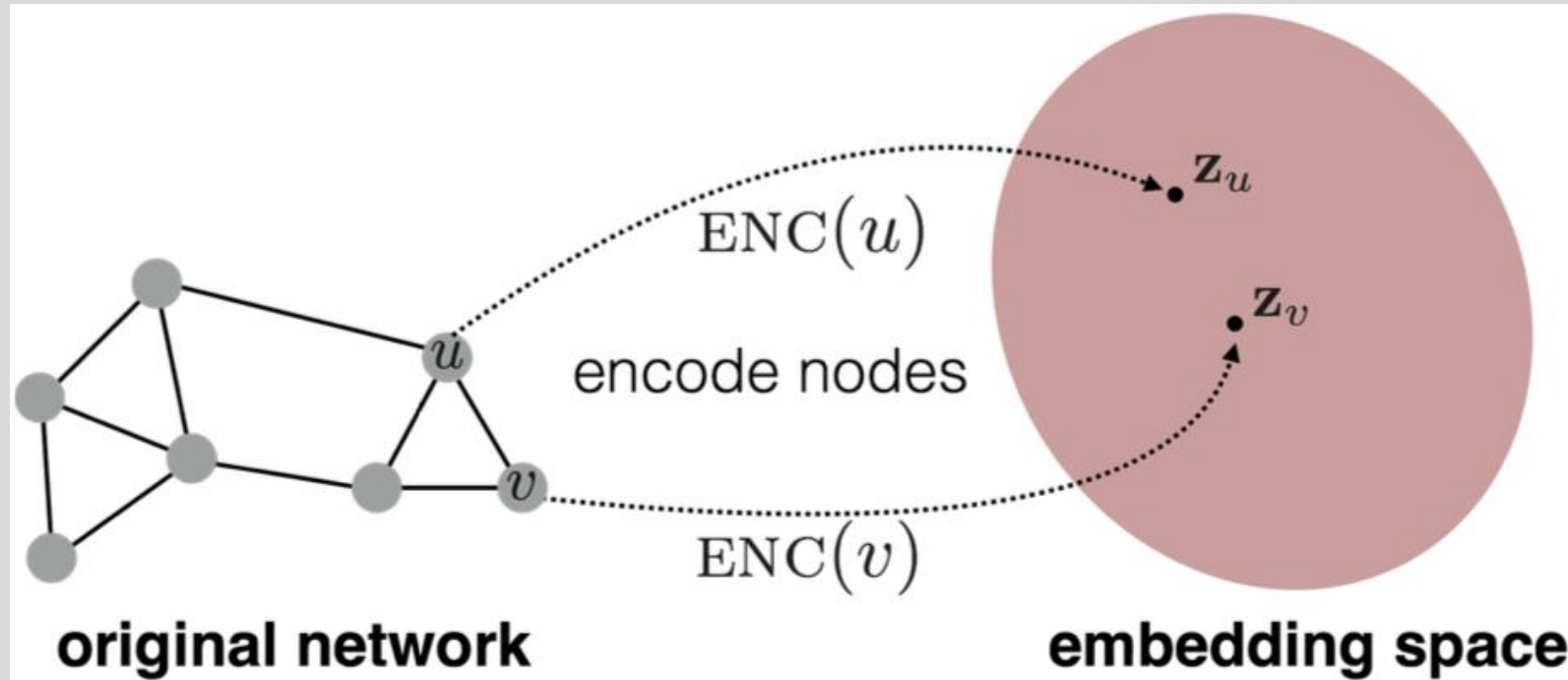
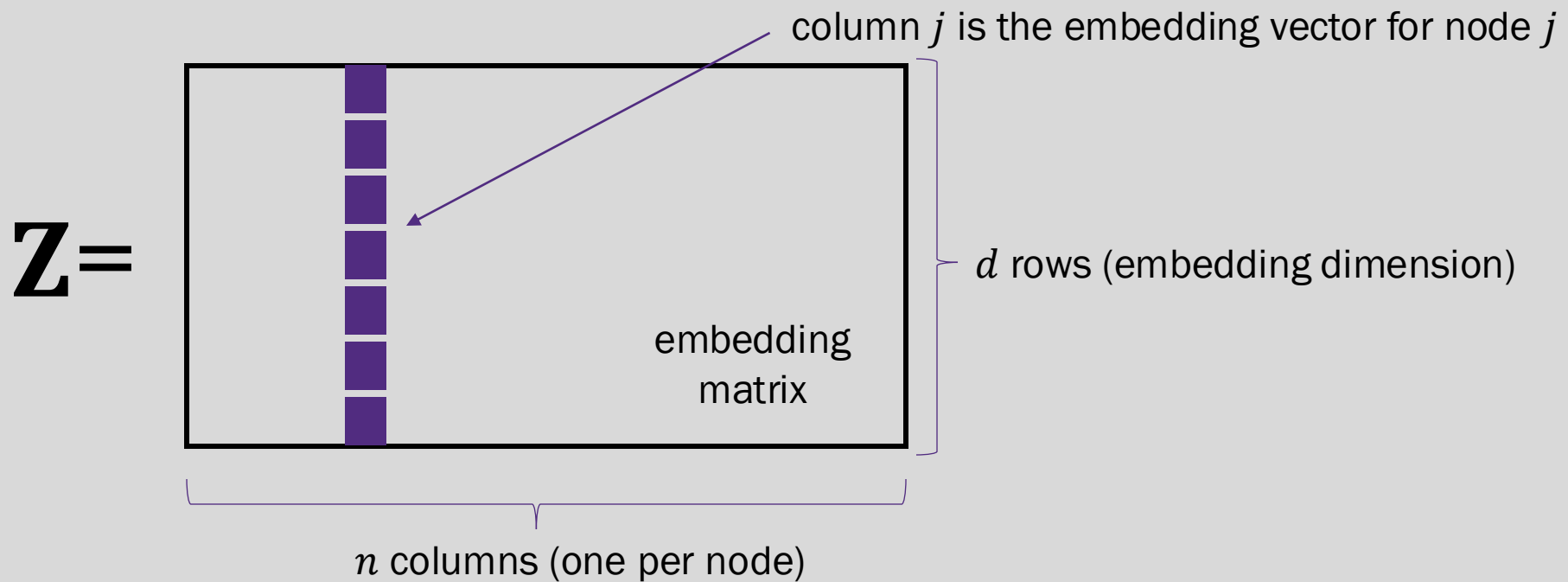


Image credit: Hamilton 2020



Shallow embeddings

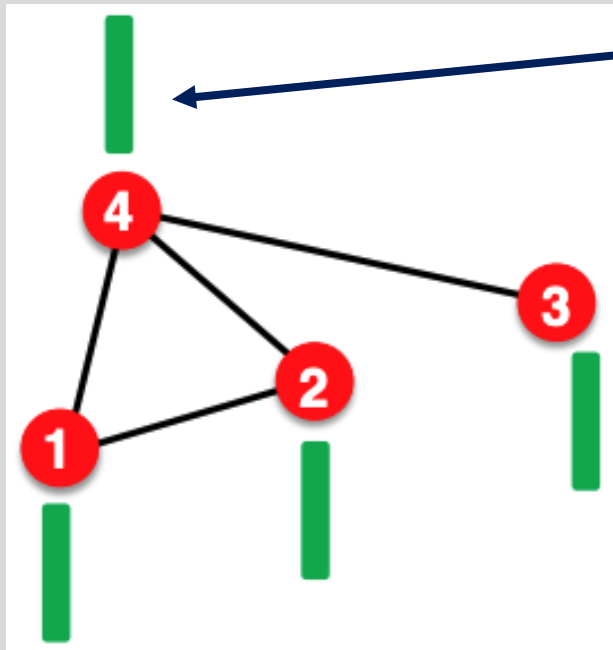
- Simplest approach, the encoder is just a lookup





Shallow embedding limitations

- Transductive (not inductive) method – cannot generate embeddings for nodes / graphs not seen in training
- No parameter sharing, $O(|V|d)$ parameters required
- Do not incorporate node features



Node feature vector

(e.g. protein properties in a protein-protein interaction graph)

Matrix factorization / random walk embeddings do not incorporate such node features

Solution: Graph neural networks

Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Deep learning meets graphs





Deep graph encoders (going beyond shallow embeddings)

- Today, we begin discussing graph neural networks
 - Field of deep learning approaches for graph data
 - A defining characteristic is the encoder

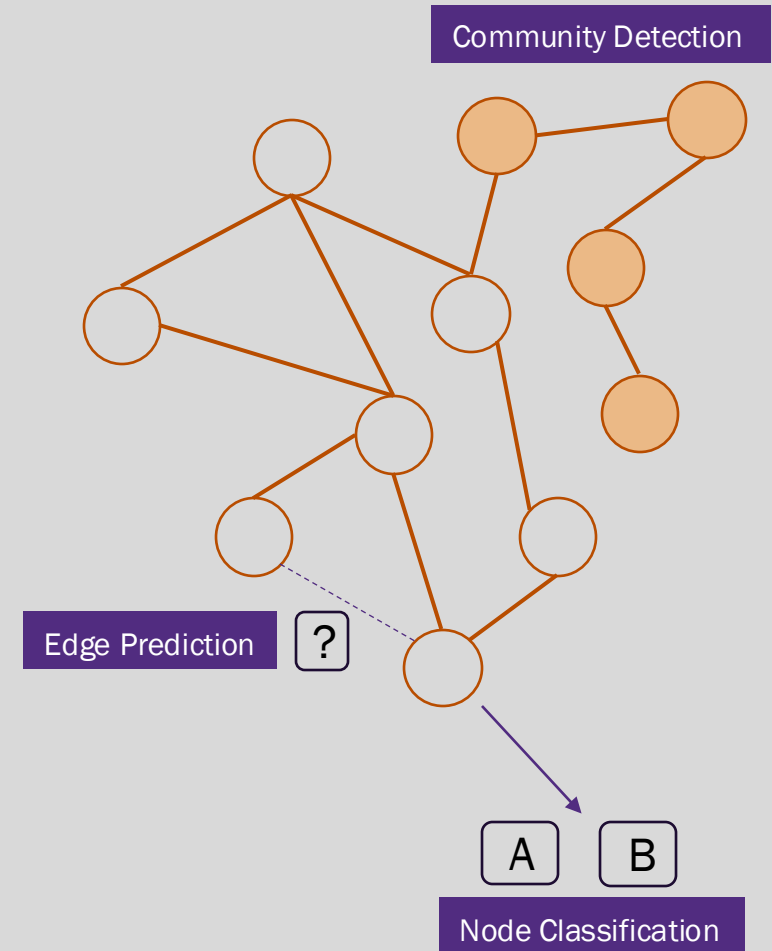
$$ENC(v) \equiv \begin{array}{l} \text{multiple iterations (layers) of} \\ \text{non-linear transformations} \\ \text{based on graph structure} \end{array}$$

- We may also use a decoder (though not always), $DEC(u, v) \approx \mathbf{S}[u, v]$ where $\mathbf{S}$ can be any node similarity function



Intra-graph machine learning tasks

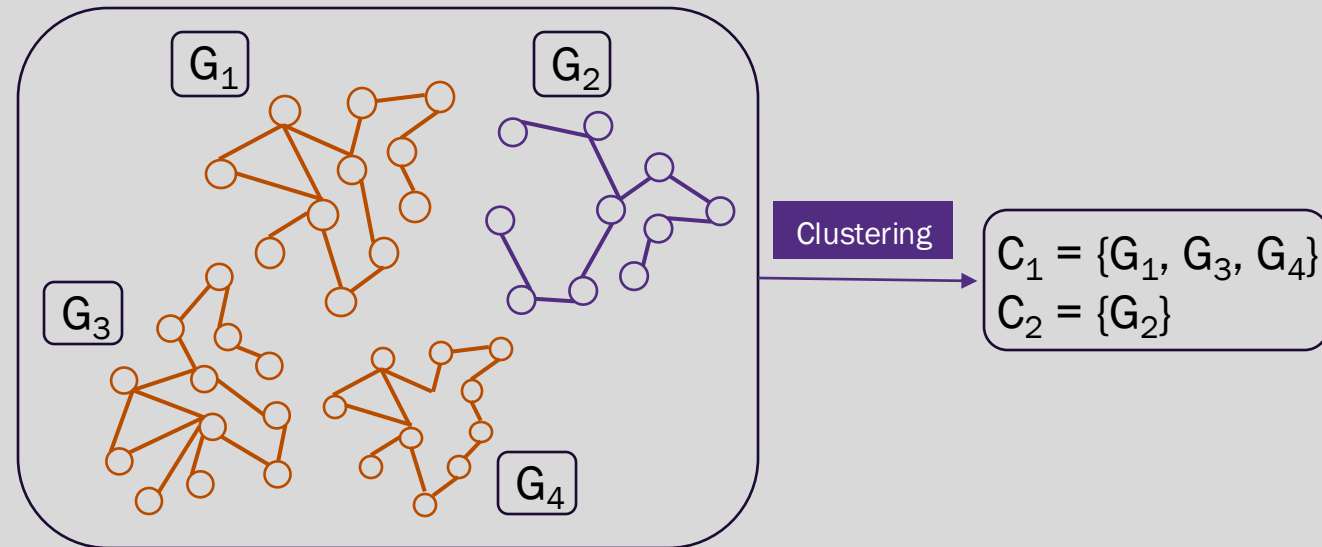
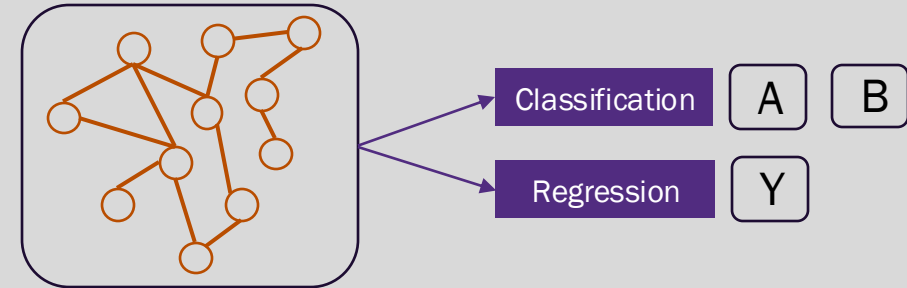
- Node classification – given information about the node (may include neighbors, relationships, attributes), infer class membership (usually an attribute)
- Edge prediction – given information about two nodes, determine if a given relation exists between them
- Community detection (node clustering) – detection of “meaningful” node groups





Whole graph and inter-graph machine learning tasks

- Whole graph classification and regression – given one entire graph, infer class membership or estimate (regress) a related numerical property
- Inter graph tasks – given multiple graphs estimate the similarity between graphs and/or identify clusters



Borrowing concepts from deep learning

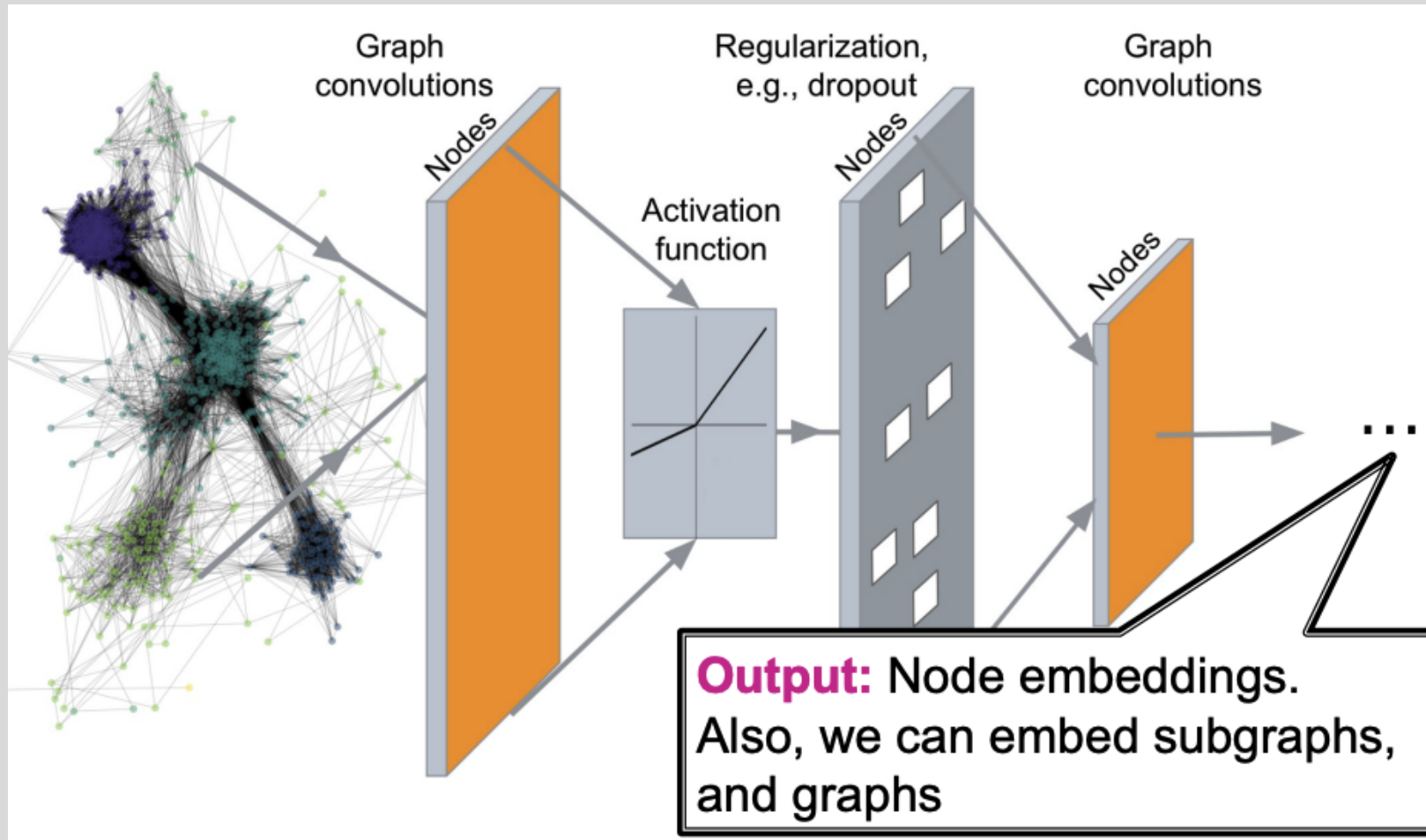
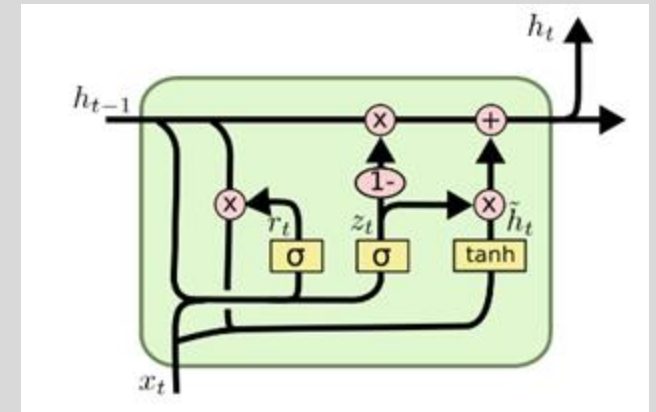
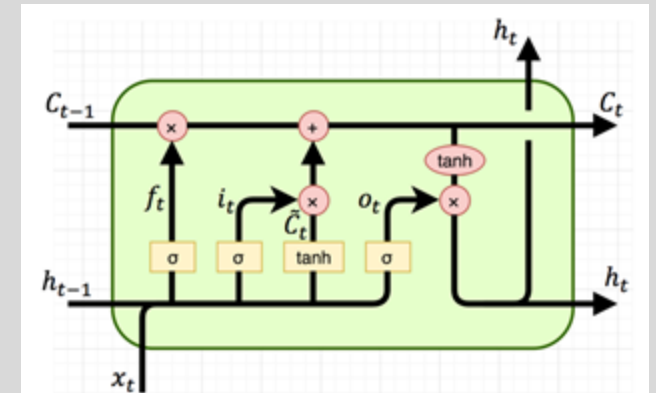
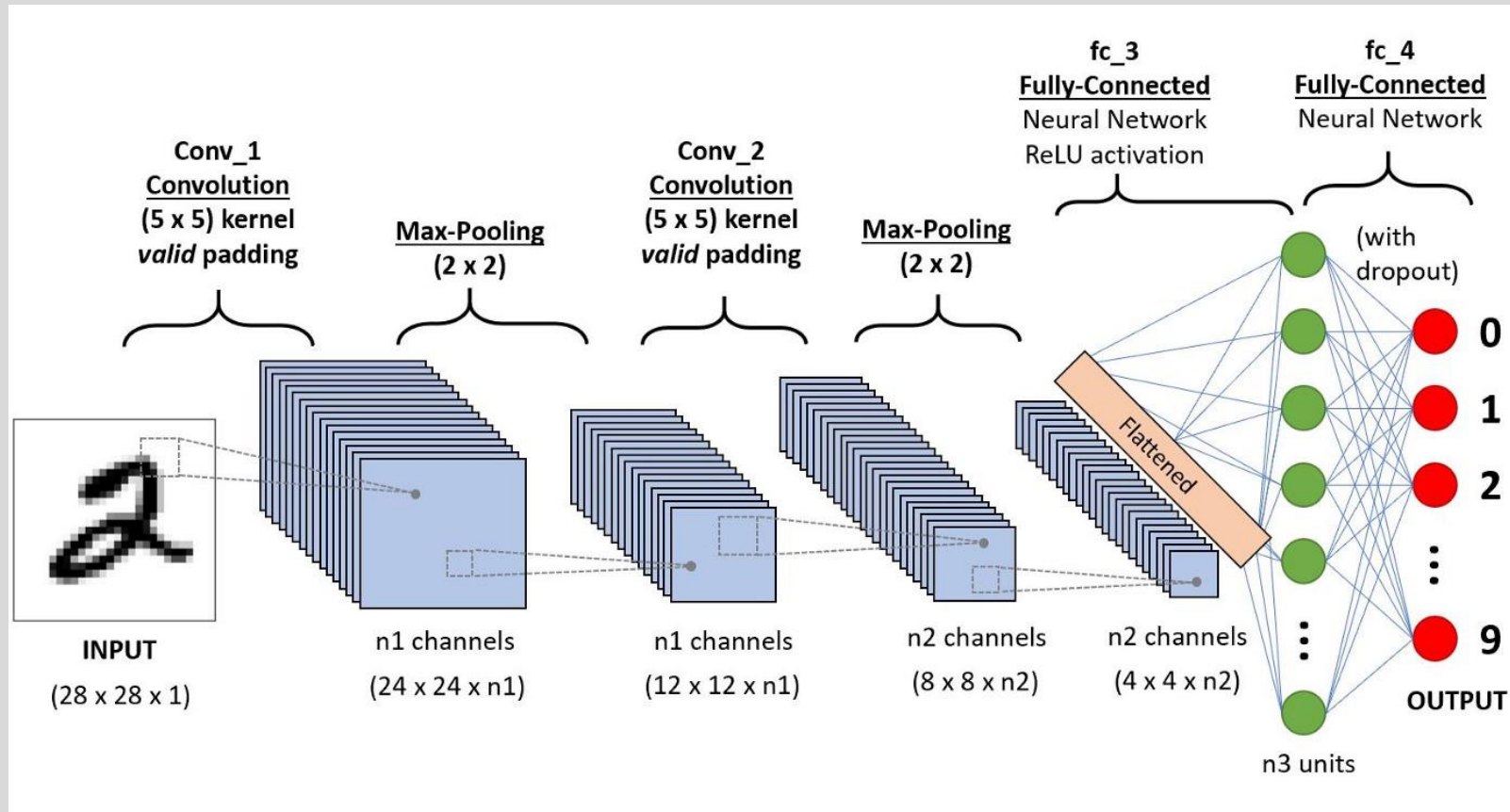


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Why can't we just use existing methods?

Standard ML methods and deep learning assume fixed-sized and ordered inputs



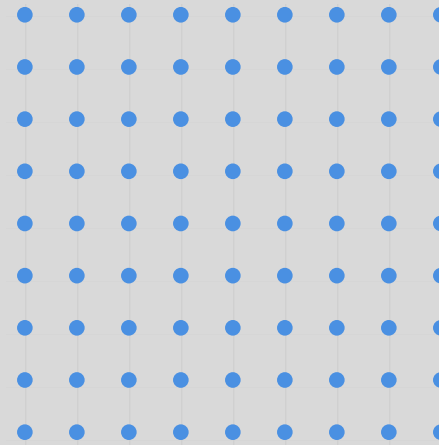


Modern ML toolbox assumes regularity

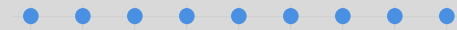
- Graphs contain arbitrary node and edge counts
- Graph nodes do not have an objective ordering
- Different node types may have different features



Graph Data



Images



Text



Review of deep learning basics

- Select a neural network model, $f_{\Theta}(\mathbf{x})$, parameterized on Θ that estimates the unknown relationship, f , between the input, $\mathbf{x}$, and an output, y .
 - f_{Θ} can be any differentiable network (linear layer, multilayer perceptron, CNN, GNN)
- Select a loss function $\mathcal{L}$ to minimize
$$\min_{\Theta} \mathcal{L}(y, f_{\Theta}(\mathbf{x}))$$
- Sample minibatches of input and output samples $\mathbf{x}, y$
- **Forward propagation:** Compute $\mathcal{L}$ from minibatch samples
- **Back-propagation:** Obtain $\nabla_{\Theta} \mathcal{L}$ using chain rule of differentiation
- Apply **stochastic gradient descent (SGD)** to update model parameters, Θ , in the negative direction of $\nabla_{\Theta} \mathcal{L}$

How can we do this for graph networks?



Graph deep learning paradigm

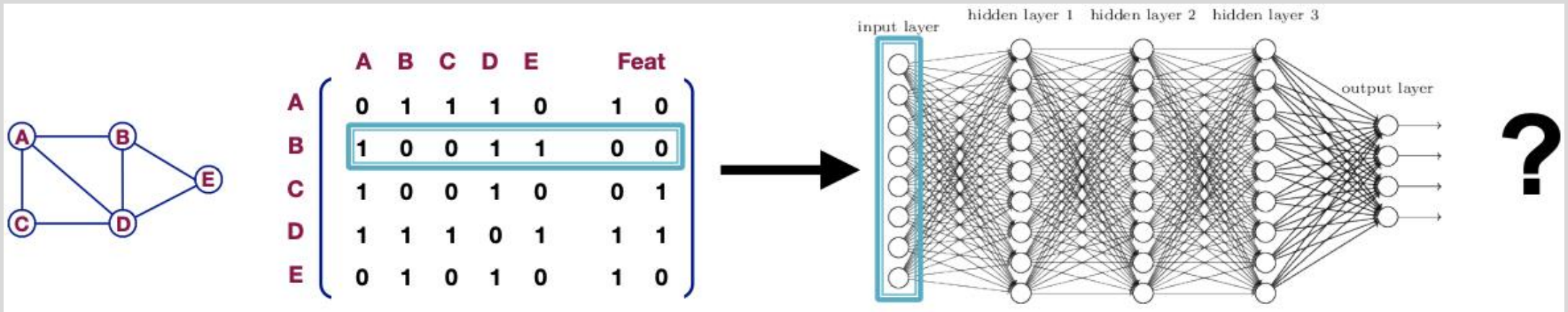
- Assume we have a graph G
- V is the set of vertices in G
- A is the adjacency matrix (assume unweighted, undirected for now)
- $X \in \mathbb{R}^{|V| \times d_l}$ is a matrix of **node features (this is new!)**
 - One d_l -dimensional feature vector per node
 - Examples of node features
 - User profile data, user image
 - Gene expression, gene functional properties
 - What if my nodes don't have features?
 - One-hot embeddings - not recommended as this makes the model transductive
 - Constant vector of 1 – model is again inductive, features will be updated (more to come)



A naïve approach to graph deep learning

- Concatenate the adjacency matrix and features into a single feature matrix
- Provide as input to a feed-forward deep neural network

Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



- Serious limitations
 - $O(|V| + d)$ parameters where d is node feature vector dimension
 - Model is transductive
 - Model is sensitive to arbitrary node ordering

What else can
we do?

Can we apply convolutional network approaches?

Original Image

1	1	1	1	1	1
1	1	1	1	1	1
0	0	1	1	0	0
0	0	1	1	0	0
0	0	1	1	0	0
0	0	1	1	0	0

Feature Map

Feature Map

6	6	6	6
4	5	5	4
2	4	4	2
2	4	4	2

Max
Pooling

Average
Pooling

Sum
Pooling

x1	x1	x1
x1	x1	x1
x0	x0	x0

Max pooling is common, but average and sum pooling are also used



Can we apply convolutional network approaches?

- Generalize convolution beyond lattices (images)
- Leverage node features

CNN on an image:

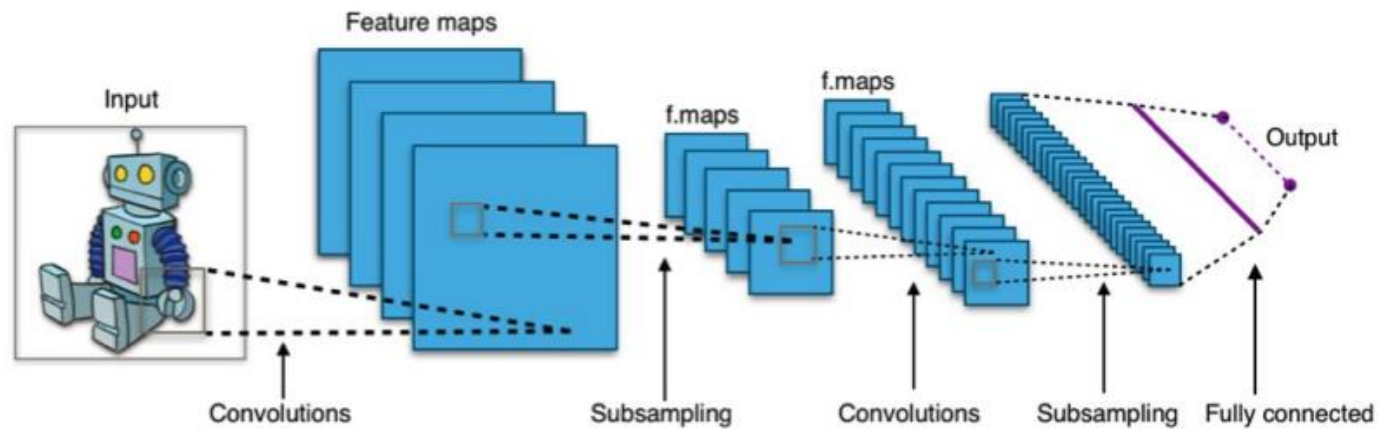
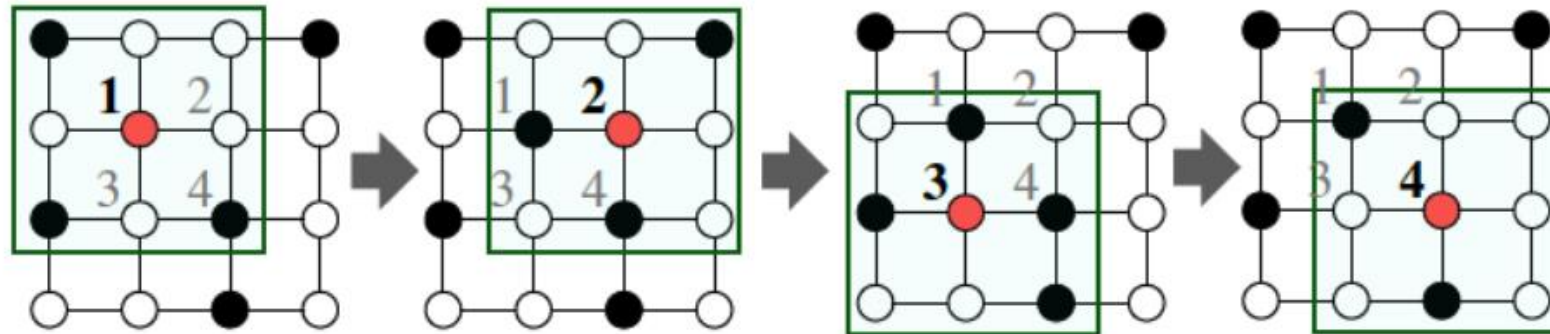


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Can we apply convolutional network approaches?

- Graphs are generally **NOT** regular lattices
- There is **no fixed reference** (locality) for a sliding window (i.e., the convolution filter)
- Graphs are **permutation invariant**

What does this mean?

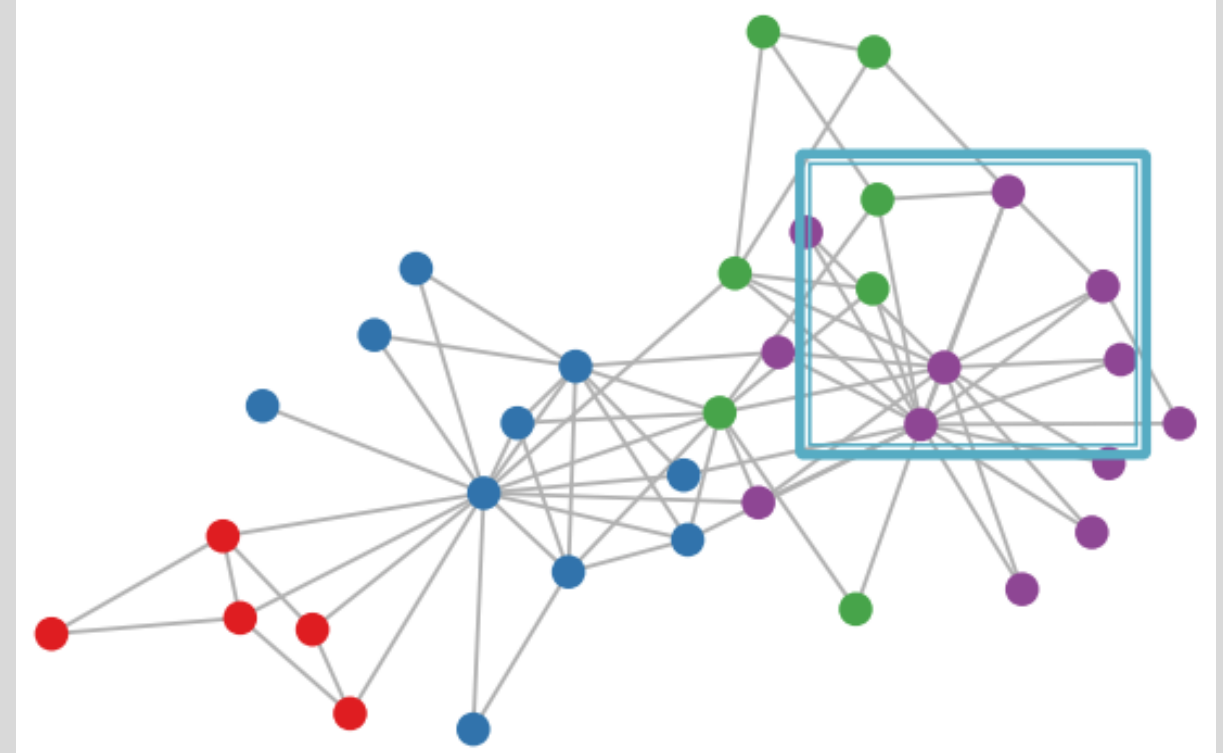


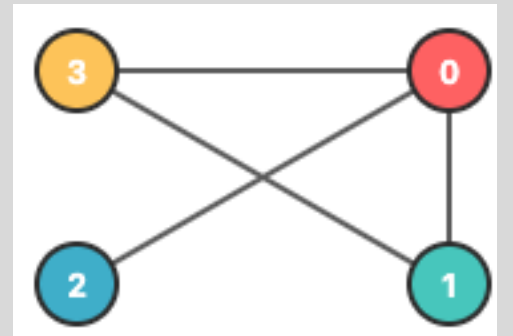
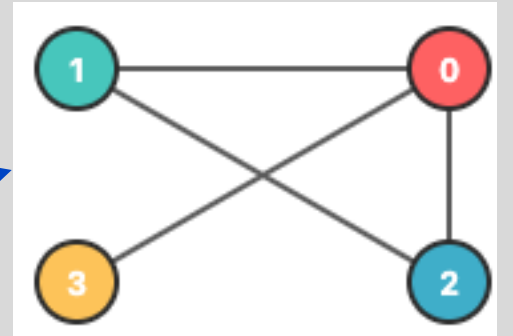
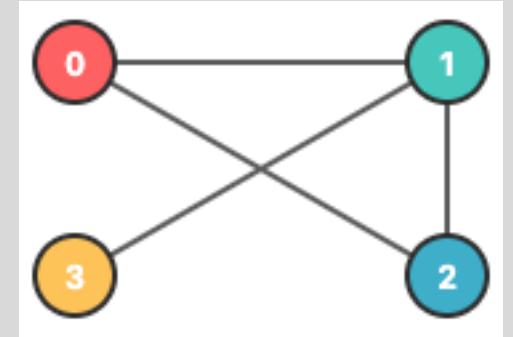
Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Permutation invariance

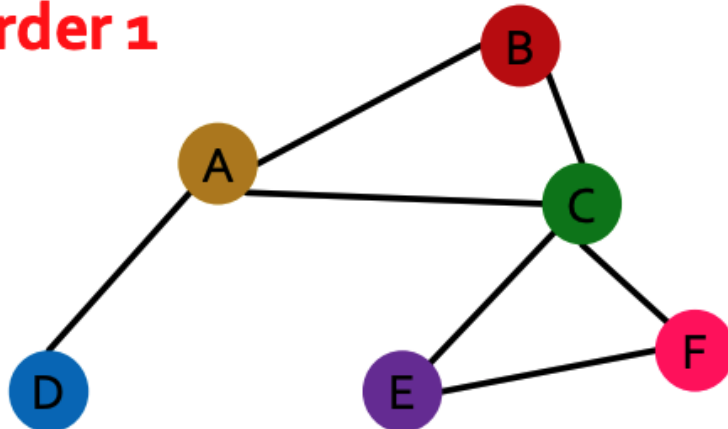
- Graphs do not have a canonical node ordering
 - We can arbitrarily assign different node orders without altering the graph structure
- Desiderata – a **graph** embedding function should produce the same result (**graph embedding**) for any node ordering

Structurally equivalent

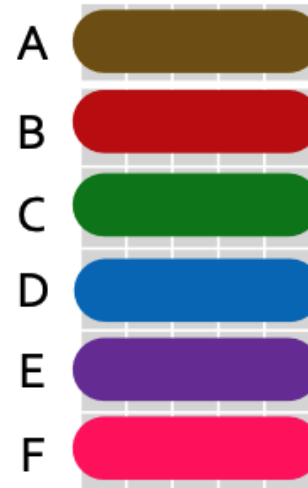


Permutation invariance

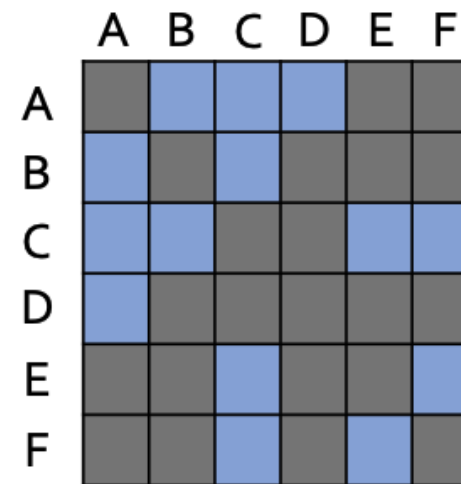
Order 1



Node features X_1



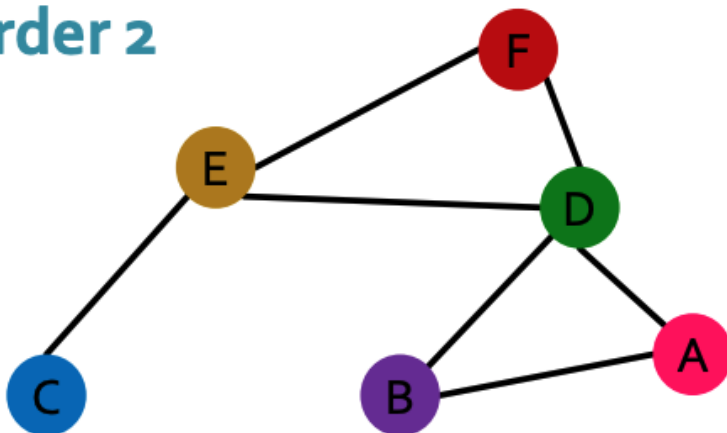
Adjacency matrix A_1



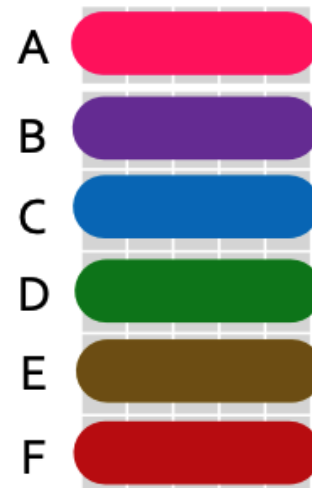
Ordering changes node position index in A

Learned graph representation should be the same for any ordering

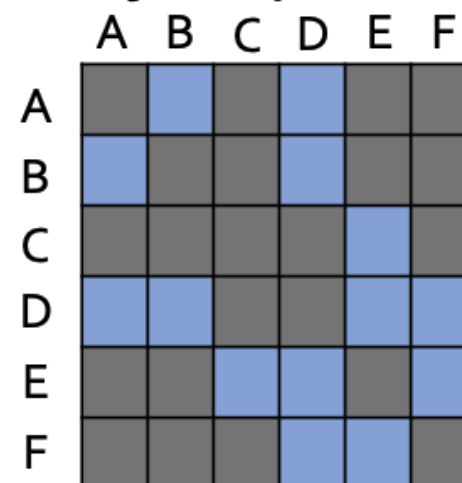
Order 2



Node features X_2



Adjacency matrix A_2





Permutation **invariance** formal definition

- Consider a function $f(\mathbf{A}, \mathbf{X}) \rightarrow \mathbb{R}^{d_{out}}$ where $\mathbf{A}$ is a graph adjacency matrix and $\mathbf{X}$ is a node feature matrix
- If $f(\mathbf{A}_i, \mathbf{X}_i) = f(\mathbf{A}_j, \mathbf{X}_j)$ for any node ordering i and j , then f is a **permutation invariant function**
 - There $|V|!$ different orderings for a graph with $|V|$ nodes. **Why?**
- Let's refine this definition for graph node embeddings
- A function $f(\mathbf{A}, \mathbf{X}): \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times d_{in}} \rightarrow \mathbb{R}^{d_{out}}$ is permutation invariant if

$$f(\mathbf{A}, \mathbf{X}) = f(\mathbf{PAP}^T, \mathbf{PX})$$

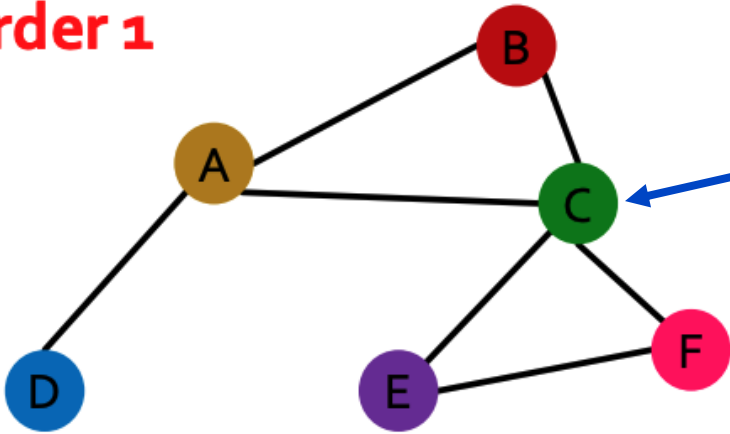
where $\mathbf{P}$ is a permutation matrix

- $\mathbf{P}$ is a matrix with exactly one 1 in each row, column (zero for other values)
- Left multiplication by $\mathbf{P}$ swaps rows
- Right multiplication by $\mathbf{P}$ swaps columns



Permutation invariance – node level

Order 1

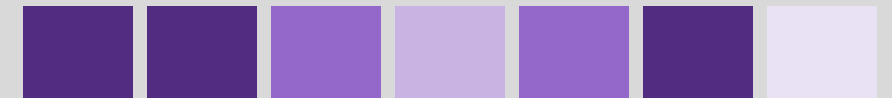


For any
target node

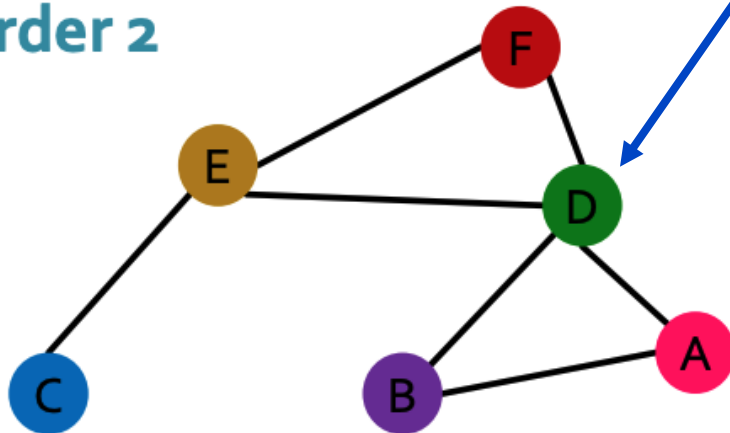
$$f(A, X) = f(PAP^T, PX)$$



Same node
embedding



Order 2

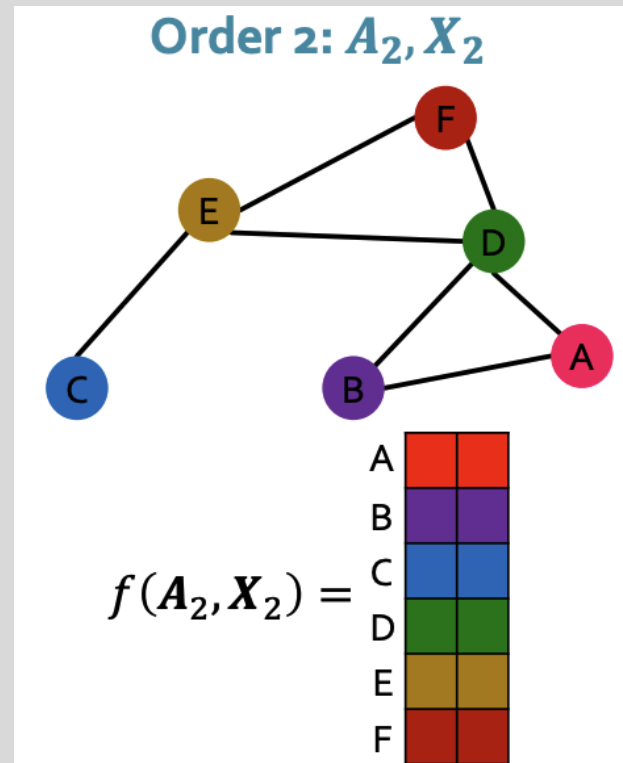
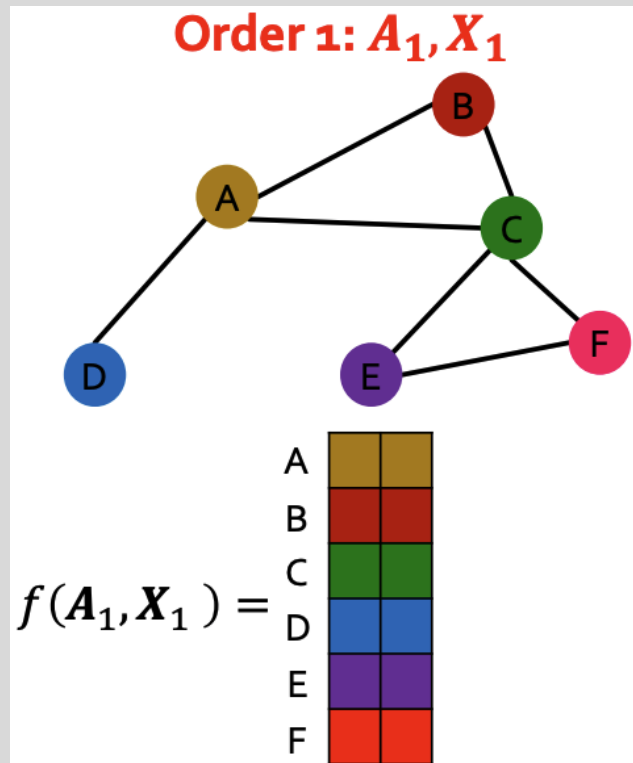


What about **graph** “permutation invariance”?
Let’s talk about ***equivariance***.



Permutation equivariance

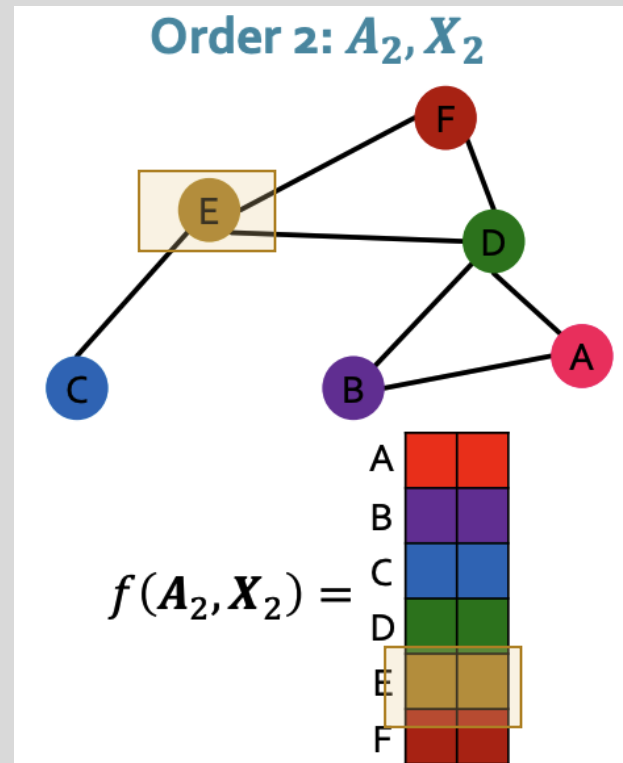
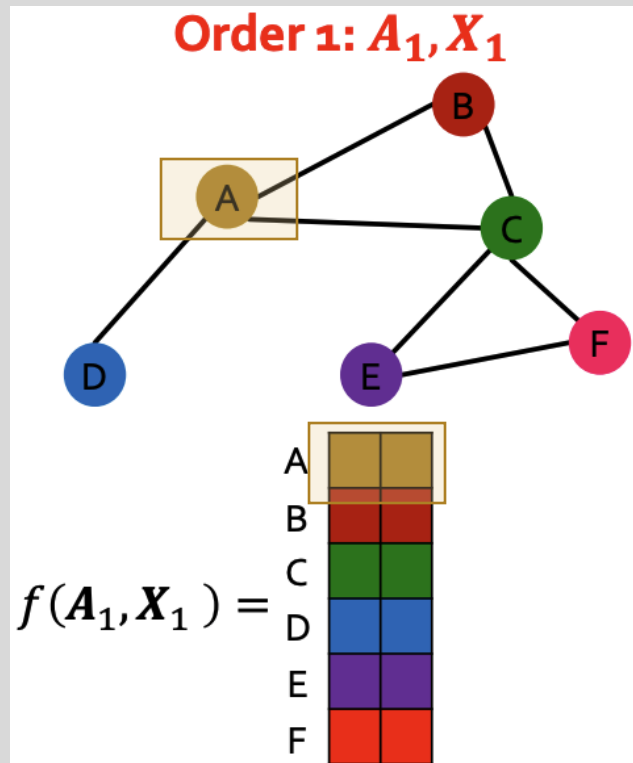
- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of G to a matrix $\mathbb{R}^{|V| \times d_{out}}$
 - Each node is mapped to a d_{out} -dimensional embedding





Permutation equivariance

- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of G to a matrix $\mathbb{R}^{|V| \times d_{out}}$
 - Each node is mapped to a d_{out} -dimensional embedding

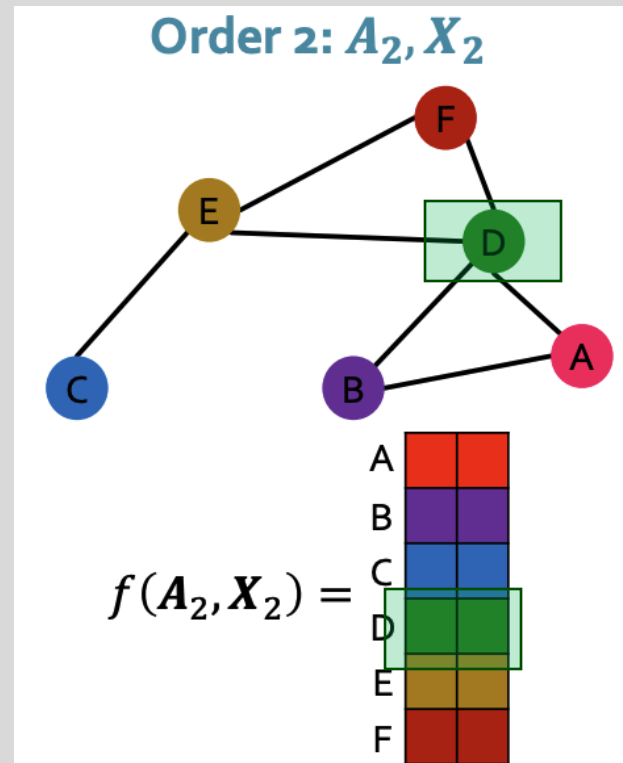
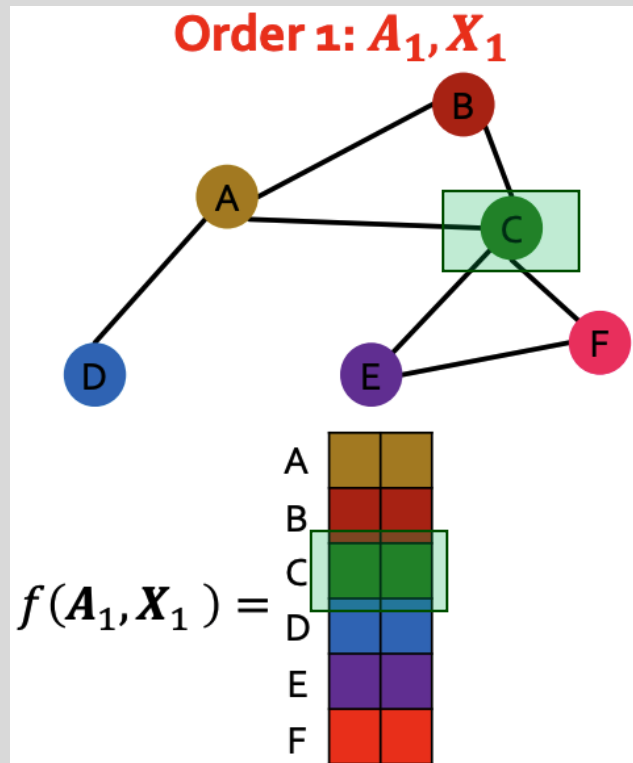


Node **A** (Order 1) is in the same position as node **E** (Order 2). It should have the same embedding (invariance)



Permutation equivariance

- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of G to a matrix $\mathbb{R}^{|V| \times d_{out}}$
 - Each node is mapped to a d_{out} -dimensional embedding

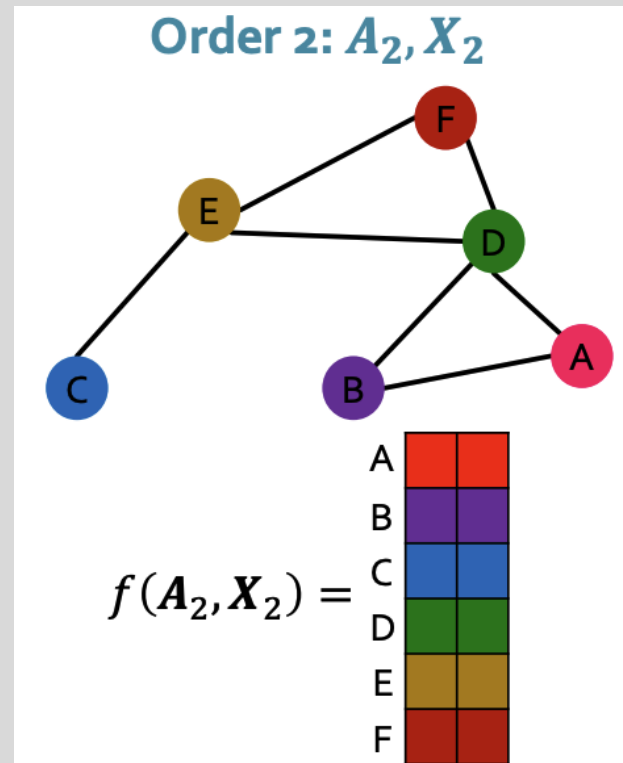
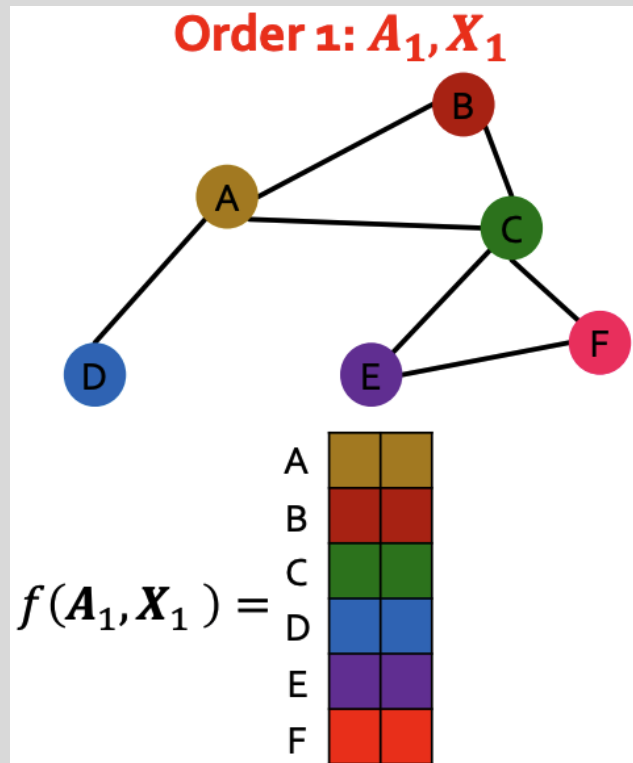


Node **C** (Order 1) is in the same position as node **D** (Order 2). It should have the same embedding (invariance)



Permutation equivariance

- As with shallow embeddings, with GNN we will learn a function, $f: ENC(v)$, that maps nodes of G to a matrix $\mathbb{R}^{|V| \times d_{out}}$
 - Each node is mapped to a d_{out} -dimensional embedding



The matrix of all node embeddings should be the same except for row shuffling (permutation)



Permutation equivariance formal definition

- Consider a function $f(\mathbf{A}, \mathbf{X}) \rightarrow \mathbb{R}^{|V| \times d_{out}}$ where $\mathbf{A}$ is a graph adjacency matrix and $\mathbf{X}$ is a node feature matrix
- If the embedding of a node (it's row in the output matrix) in the same graph position is unchanged for any node ordering, f is **permutation equivariant**
- Formally, a function $f(\mathbf{A}, \mathbf{X}): \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times d_{in}} \rightarrow \mathbb{R}^{d_{out}}$ is **permutation equivariant** if

$$\mathbf{P}f(\mathbf{A}, \mathbf{X}) = f(\mathbf{PAP}^T, \mathbf{PX})$$

where $\mathbf{P}$ is a permutation matrix

f outputs a node embedding matrix which is permuted in the same manner as $\mathbf{A}$

<https://ml4g.masinolab.com/topics/gnns/gnn-permutation-demo.html>



Permutation invariance and equivariance

- Permutation invariant – permute the input, the output is unchanged

$$f(\mathbf{A}, \mathbf{X}) = f(\mathbf{PAP}^T, \mathbf{PX})$$

- Permutation equivariance - permute the input, the output is permuted in the same way

$$\mathbf{P}f(\mathbf{A}, \mathbf{X}) = f(\mathbf{PAP}^T, \mathbf{PX})$$



GNNs chain permutation invariant and equivariant operations

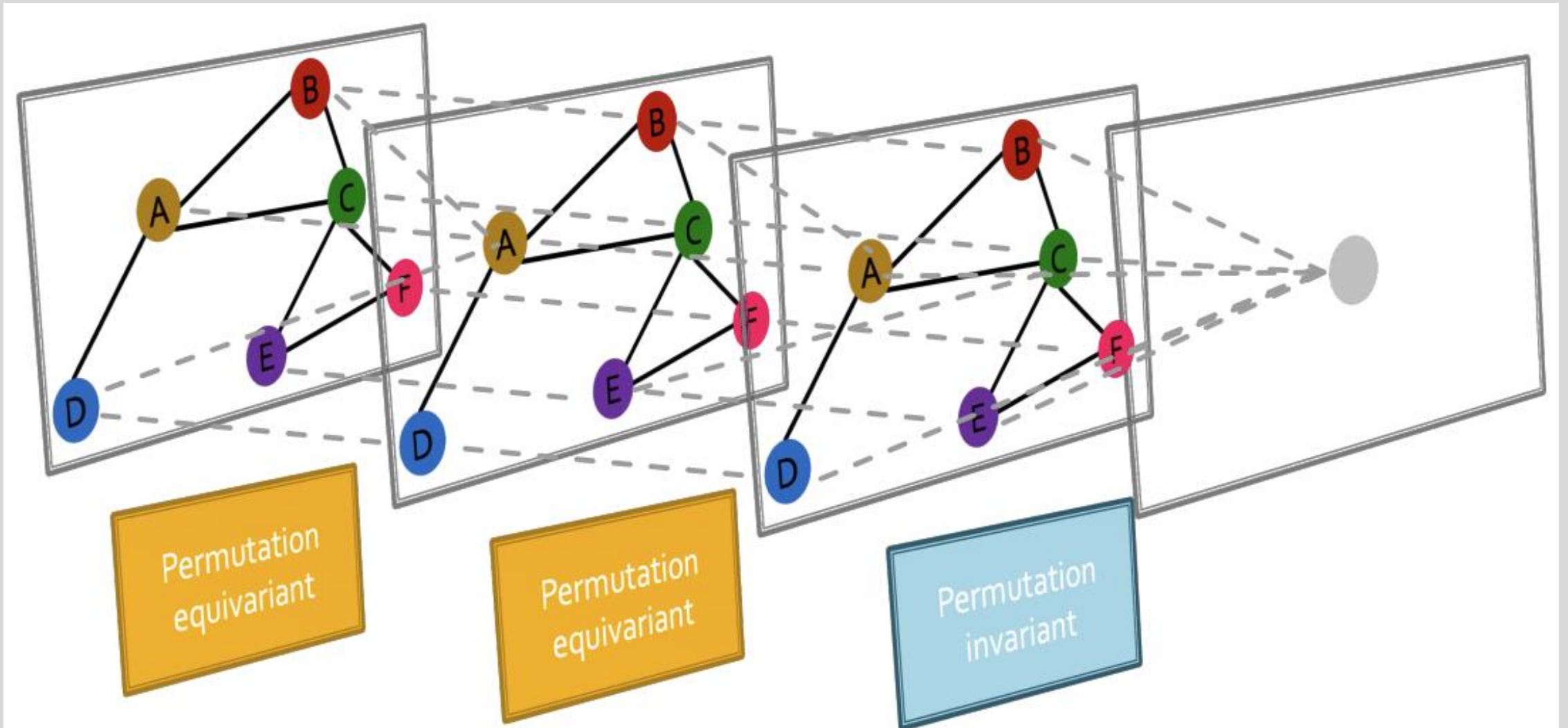


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Standard neural networks are permutation sensitive

Are common neural network architectures (linear layers, MLPs, CNNs, RNNs) permutation invariant / equivariant? **NO!!!**

Changing the input order changes the output. Why?

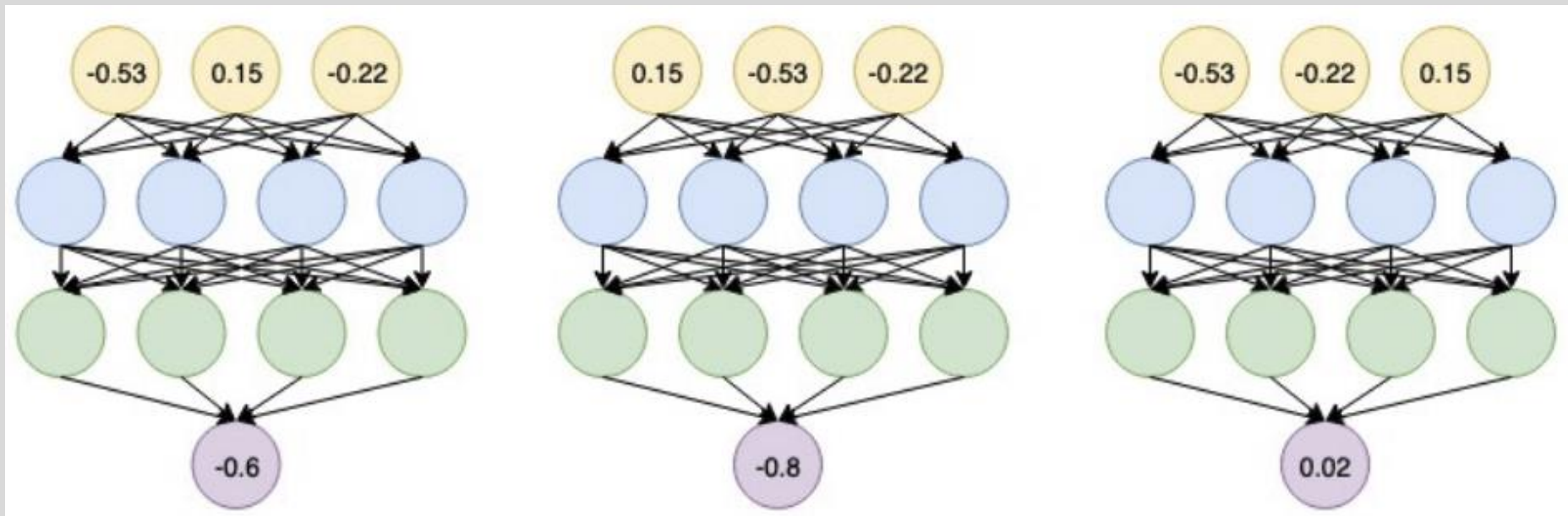


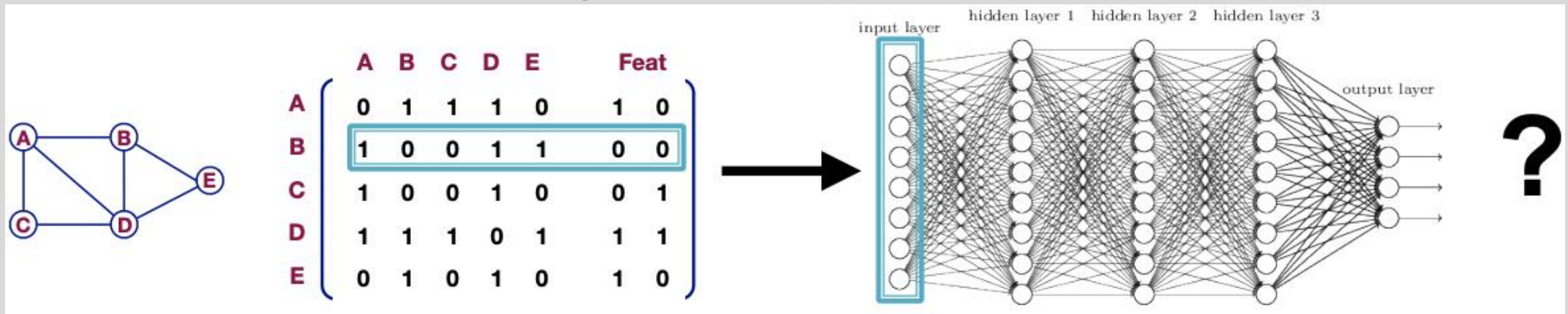
Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Standard neural networks are permutation sensitive

Are common neural network architectures (linear layers, MLPs, CNNs, RNNs) permutation invariant / equivariant? **NO!!!**

Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



This is why the naïve approach fails for graphs.



Graph convolution networks





Neighbor node aggregation

- Generate node embeddings based on local network neighborhood

Input graph

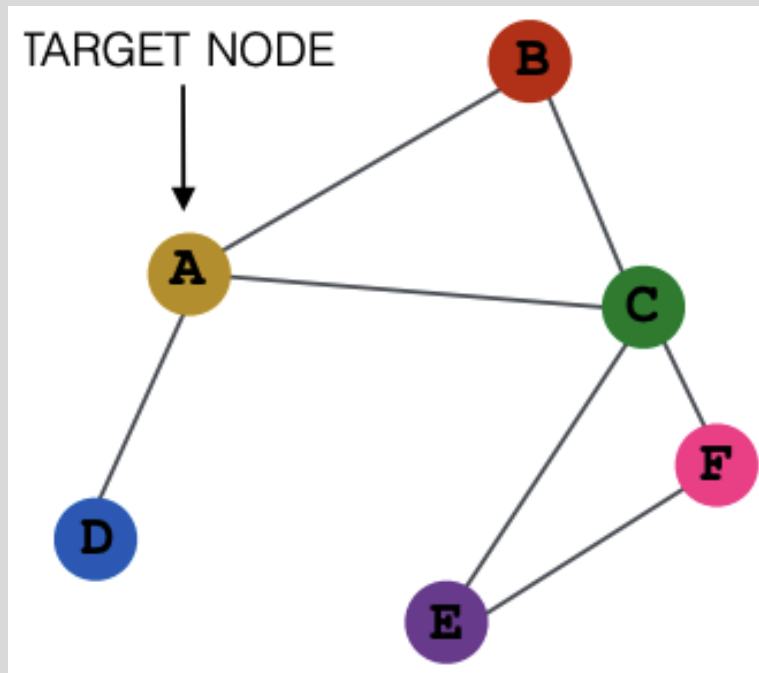
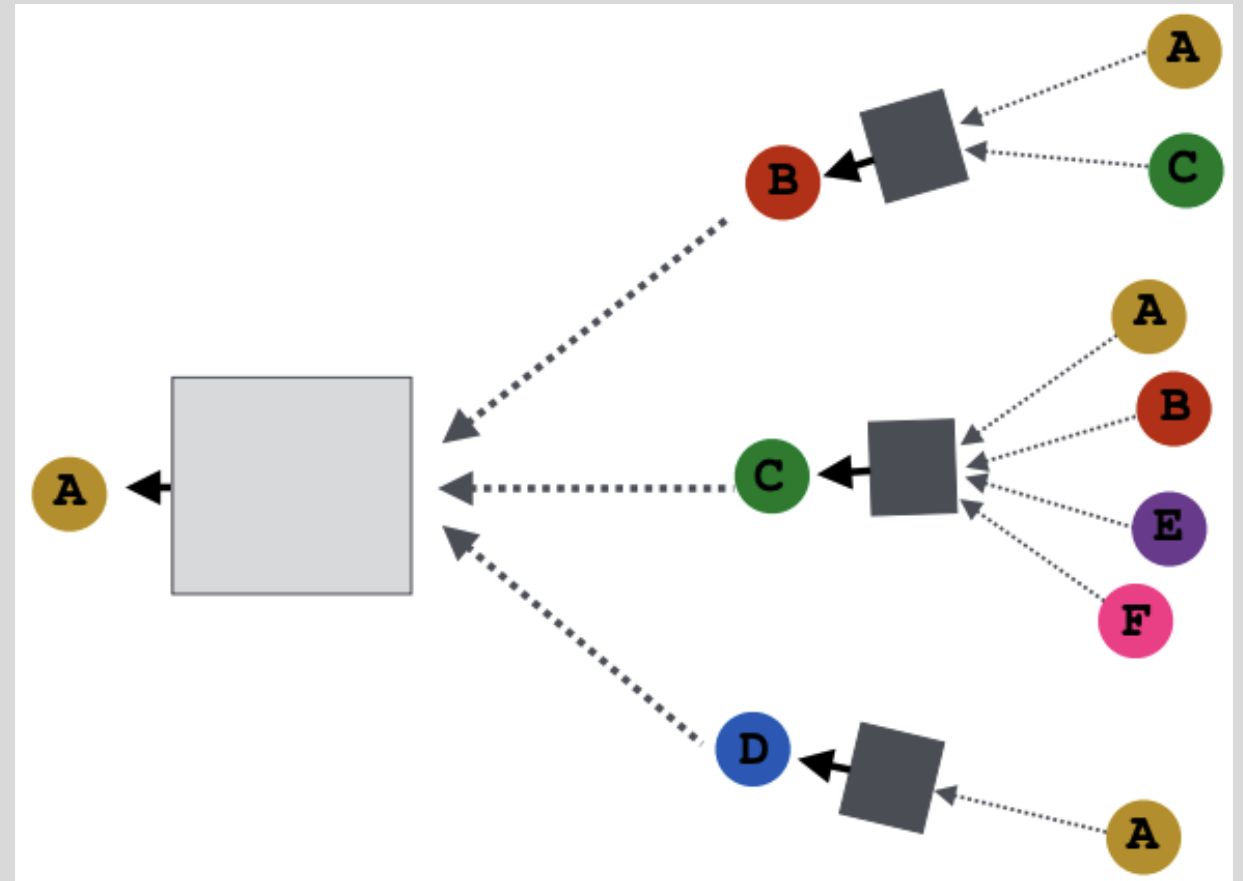


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>





Neighbor node aggregation

- Generate node embeddings based on local network neighborhood

Input graph

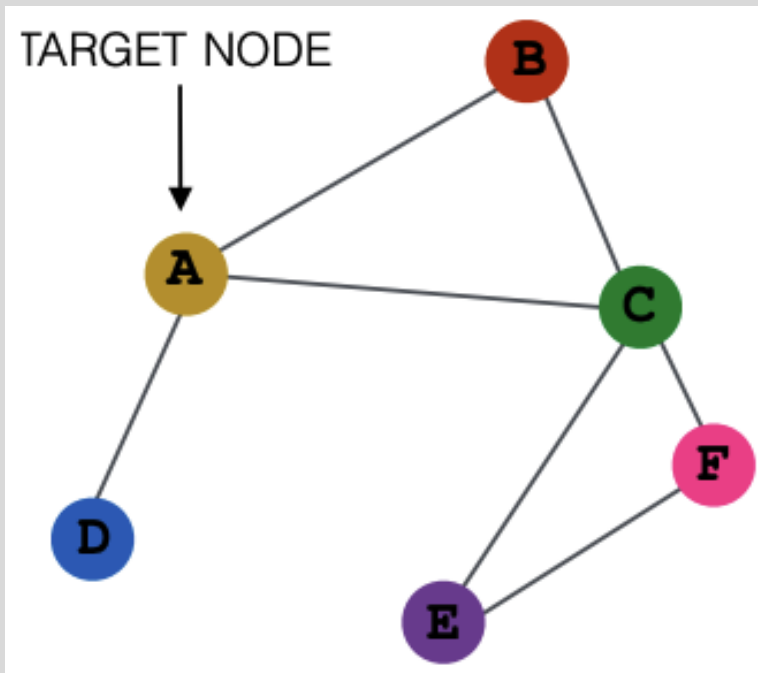
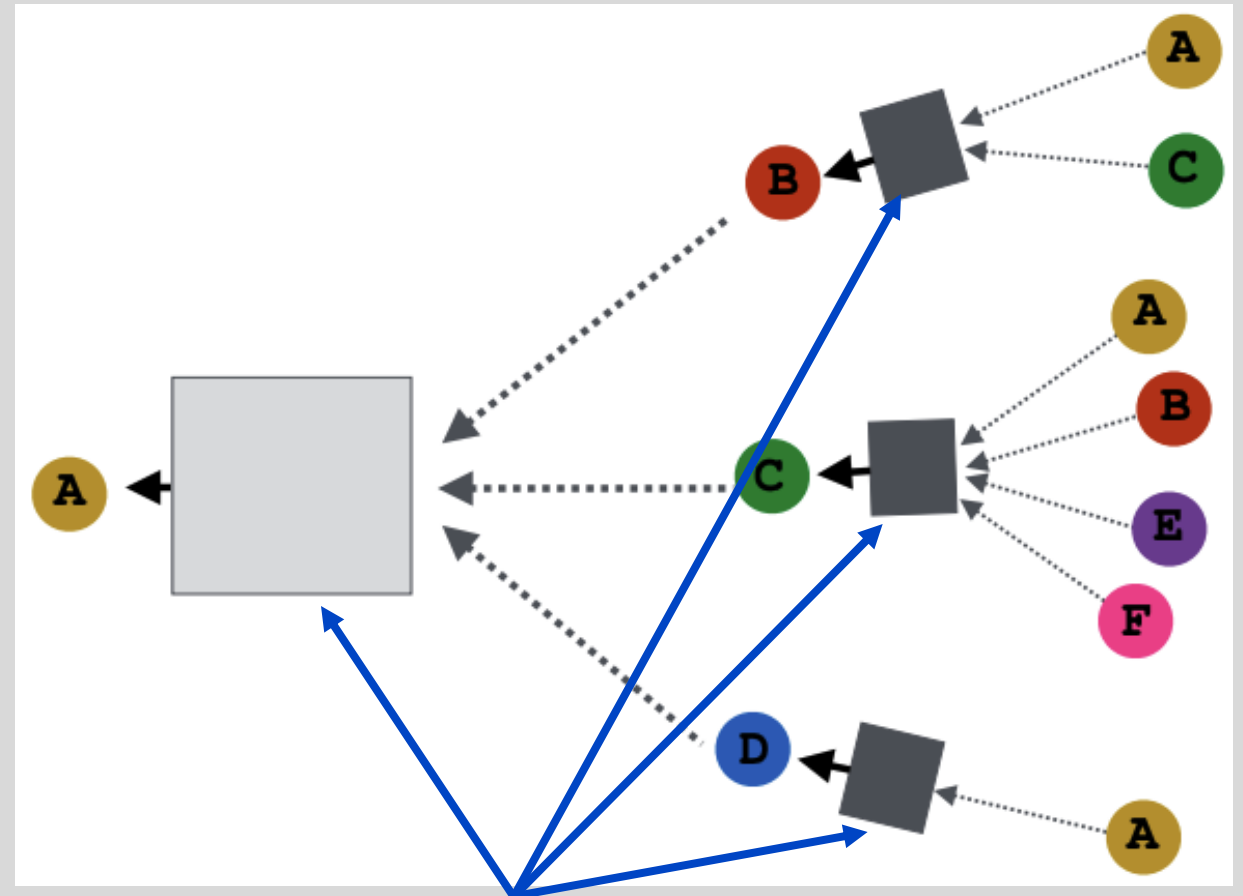


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Neural networks



Neighbor node aggregation

- Every neighborhood defines a computation graph

Each node has a unique computation graph based on its neighborhood

Input graph

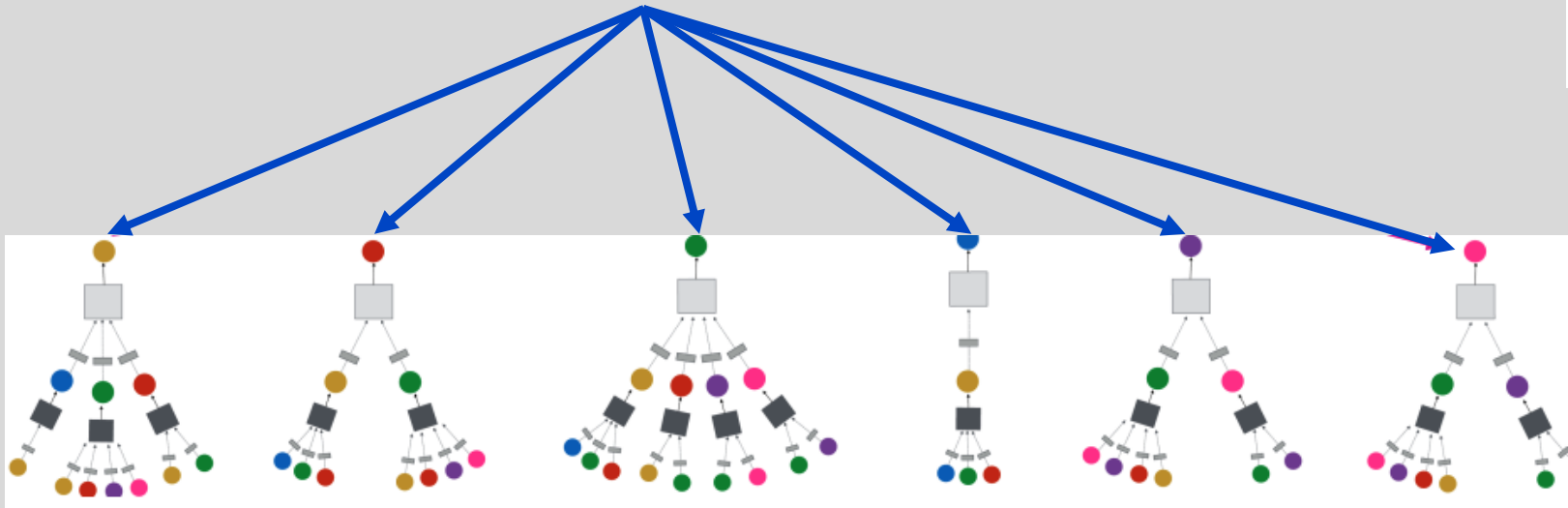
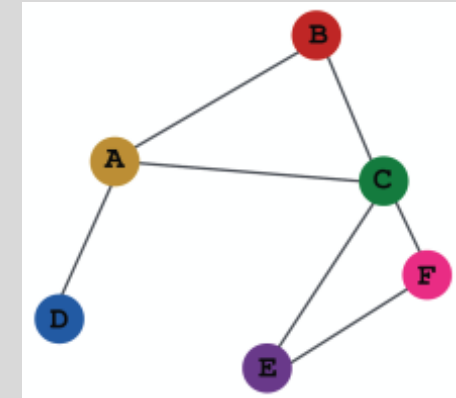


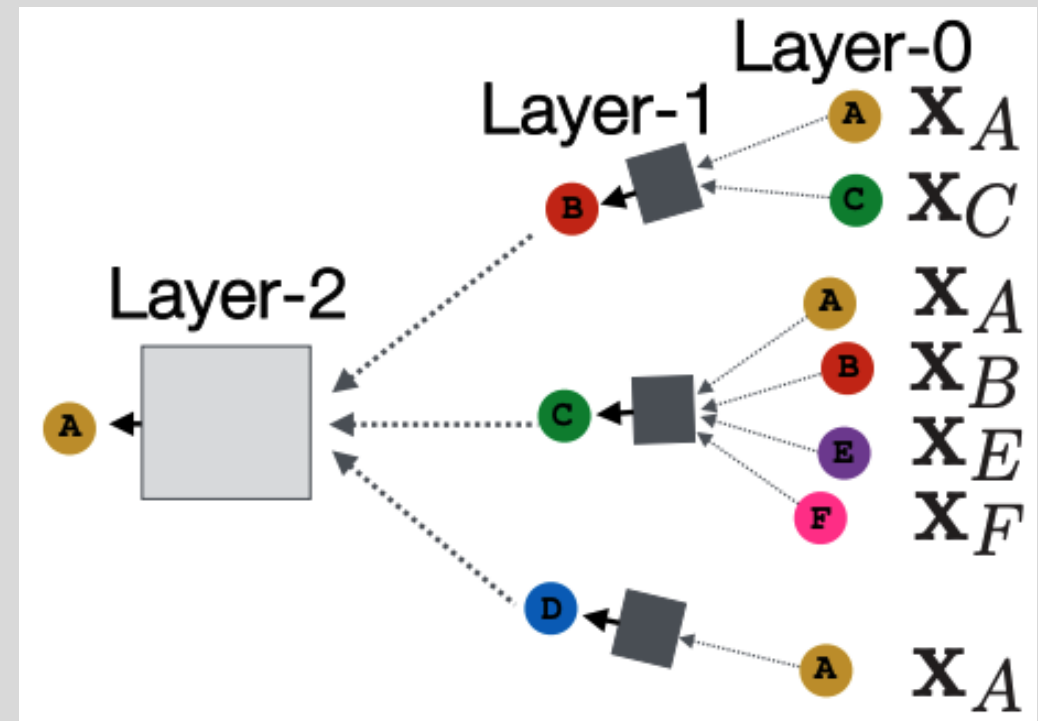
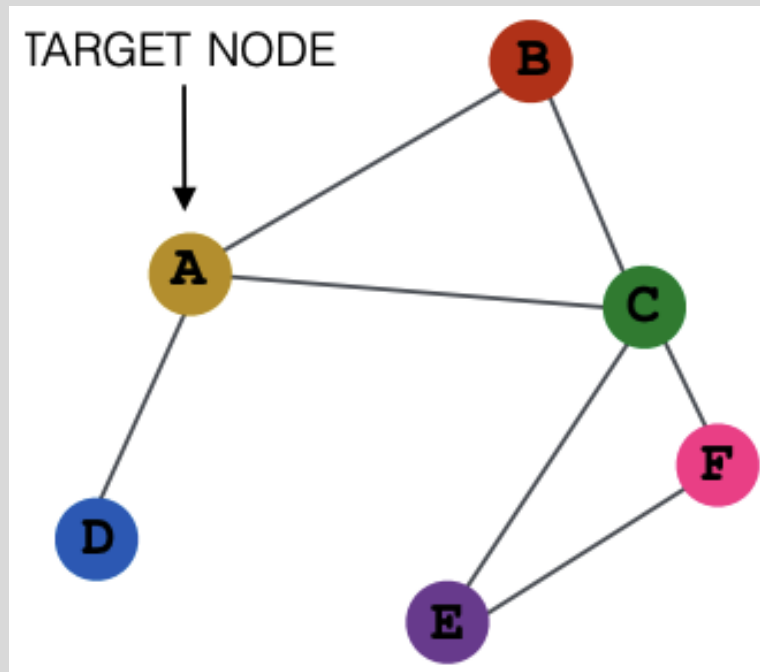
Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Neighbor node aggregation and layers

- Models can be of arbitrary depth
 - Nodes have embeddings at each layer
 - Layer-0 embedding of node v is its input vector, $\mathbf{x}_v$
 - Layer-k embedding gets information from nodes that are k hops (edges) away

Input graph

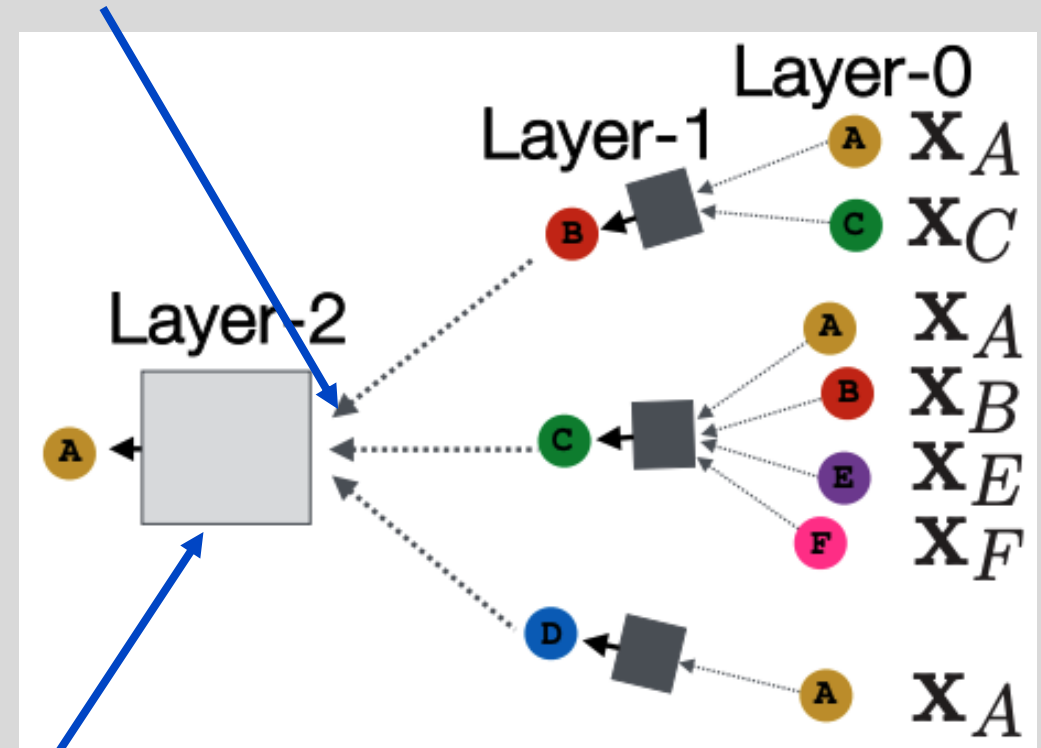
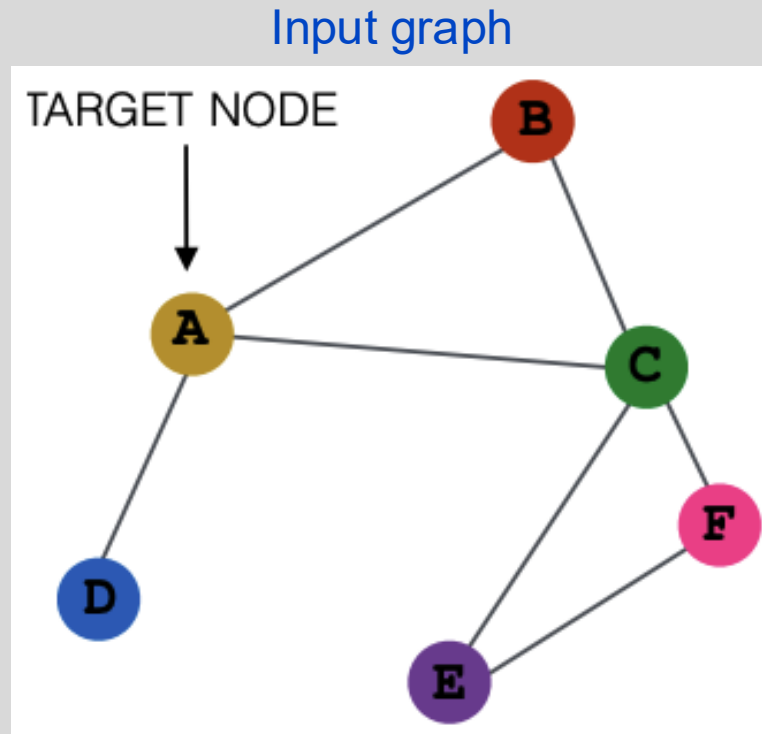




Neighbor node computation

- **One approach:** Average the *messages* (feature vectors) from node neighbors and then apply a neural network

(1) message averaging



(2) neural network



Neighbor node computation

- **Self loops:** To reduce loss of self information, self loops are added so that a node's own feature vector is used in its update

Input graph

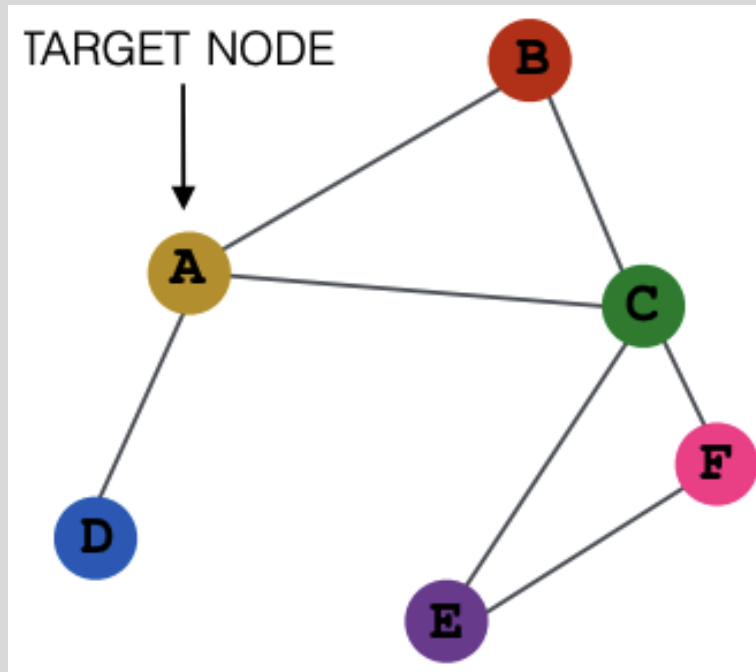
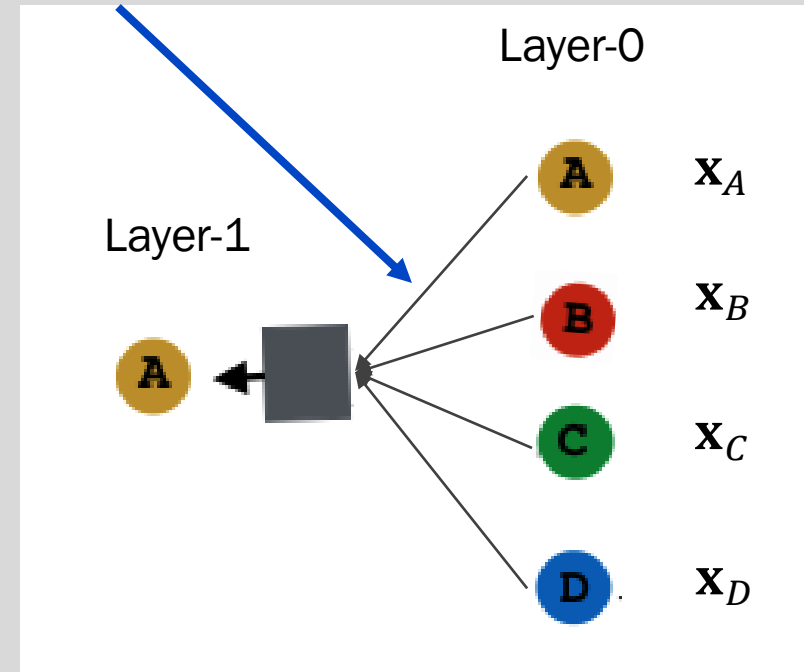


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>

(1) self loop





The math of graph convolution networks

- Consider a single node v with neighbors in $N(v)$
- Let $\mathbf{h}_v^k$ be the latent representation of v at layer k
- $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the **input feature vector** for v
- At successive layers, where $k \in [0, \dots, K - 1]$, K is the number of layers and σ is a non-linear activation

$$h_v^{(k+1)} = \sigma \left(\boxed{W^{(k)}} \sum_{u \in N(v) \cup \{v\}} \frac{h_u^{(k)}}{|N(v)|} \right)$$

- $h_v^{(K)} = \boxed{\mathbf{z}_v}$
- Learnable weight matrix
- Final node embedding

We can input the final embeddings into any loss function and learn the weight parameters using back propagation and SGD



The math of graph convolution networks in matrix form

- It will be more efficient if we can put the update equations in matrix form and leverage sparse matrix operations

- Let $\mathbf{H}^{(k)} = \begin{bmatrix} h_1^k \\ \vdots \\ h_{[V]}^k \end{bmatrix}$ be the matrix of latent node representations $\forall v \in G$

- Then letting $\mathbf{A}_{v,:}$ be the v^{th} row of the adjacency matrix (with $a_{v,v} = 1$):

$$\sum_{u \in N(v) \cup \{v\}} h_u^{(k)} = \mathbf{A}_{v,:} \mathbf{H}^{(k)}$$

- Now, let $\mathbf{D}$ be a diagonal matrix with diagonal entries $\mathbf{D}_{v,v} = \text{deg}(v) = |N(v)|$
- Then $\mathbf{D}^{-1}$ is also diagonal with entries $\mathbf{D}_{v,v}^{-1} = 1/|N(v)|$
- Finally, we can write the update function across all nodes as

$$\sum_{u \in N(v) \cup \{v\}} \frac{h_u^{(k)}}{|N(v)|} \Rightarrow \mathbf{H}^{(k+1)} = \mathbf{D}^{-1} \mathbf{A} \mathbf{H}^{(k)}$$



The math of graph convolution networks in matrix form

- From the previous result, we can rewrite the update function over all nodes as:

$$\mathbf{H}^{(k+1)} = \sigma \left(\boxed{\tilde{\mathbf{A}} \mathbf{H}^{(k)}} \mathbf{W}^{(k)T} \right)$$

where $\tilde{\mathbf{A}} = \boxed{\mathbf{D}^{-1}} \mathbf{A}$

normalization

neighborhood
aggregation

- $\tilde{\mathbf{A}}$ is sparse which allows computationally efficient sparse matrix multiplication
- Not all GNNs can be expressed in matrix form (depends on the neighborhood aggregation function)



Node embeddings are permutation invariant

The GCN that computes a given node's embedding is **independent of the ordering of the node neighbors** due to NNet weight sharing and neighbor averaging

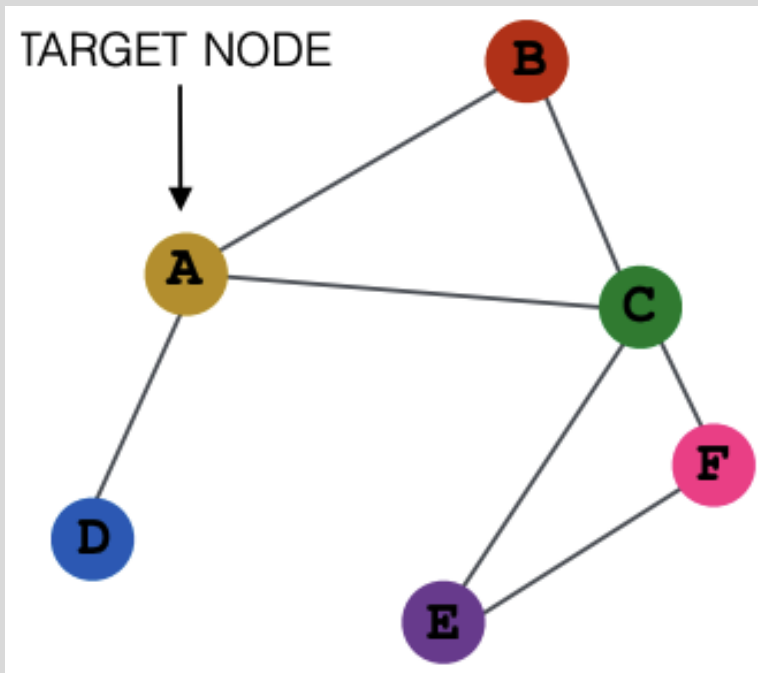
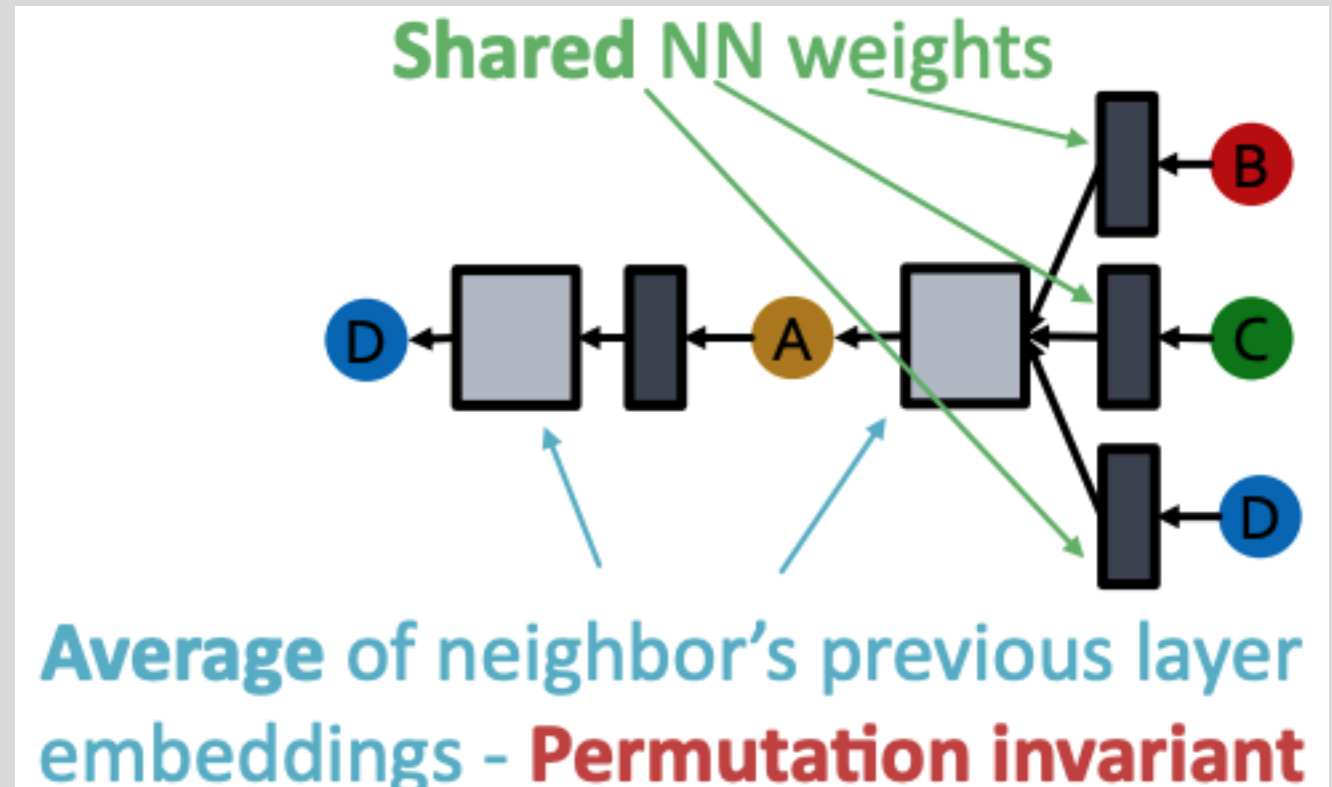


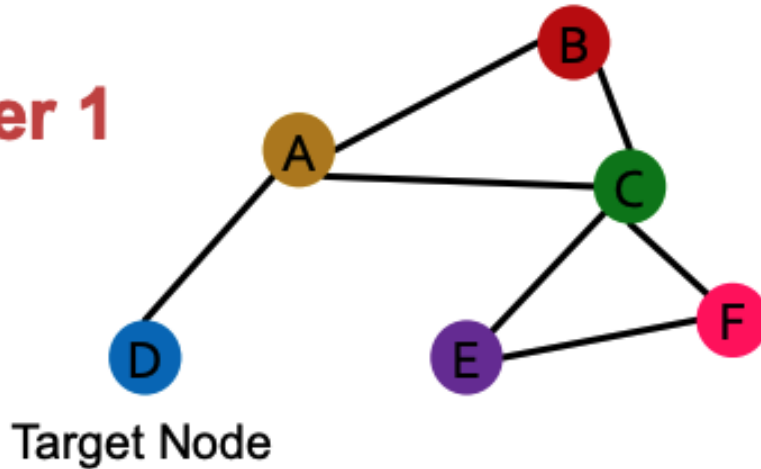
Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



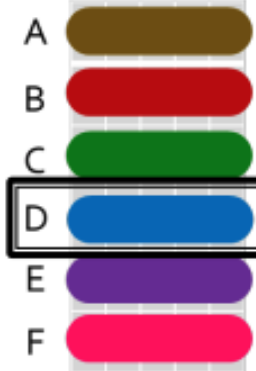


Embedding matrix of all nodes is permutation equivariant

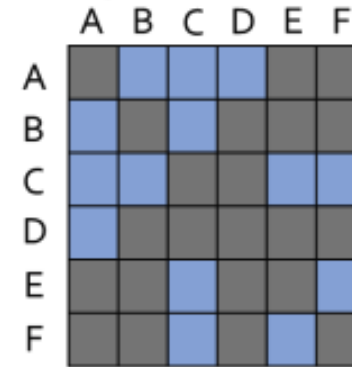
Order 1



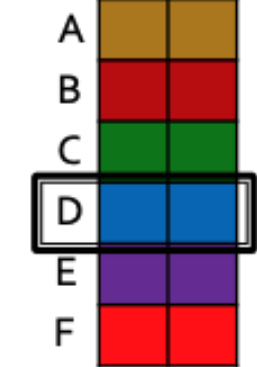
Node feature X_1



Adjacency matrix A_1

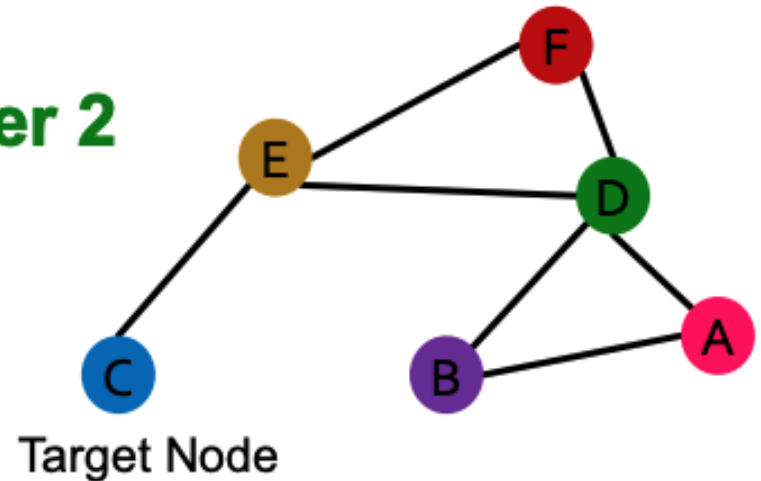


Embeddings H_1

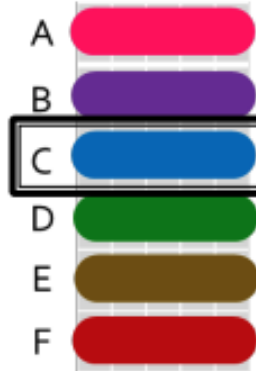


Permute the input, the output also permutes accordingly - permutation equivariant

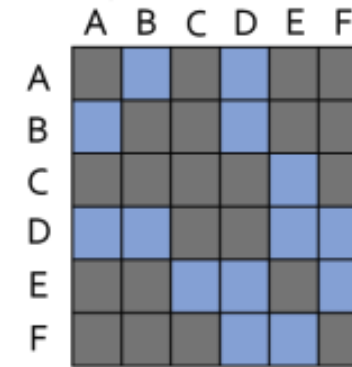
Order 2



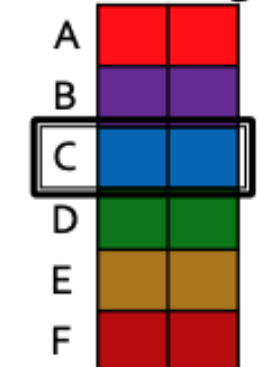
Node feature X_2



Adjacency matrix A_2



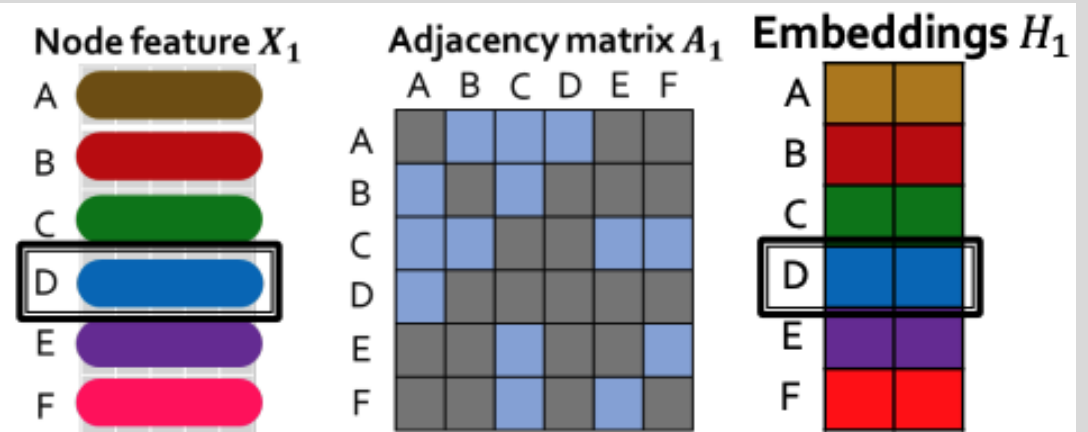
Embeddings H_2



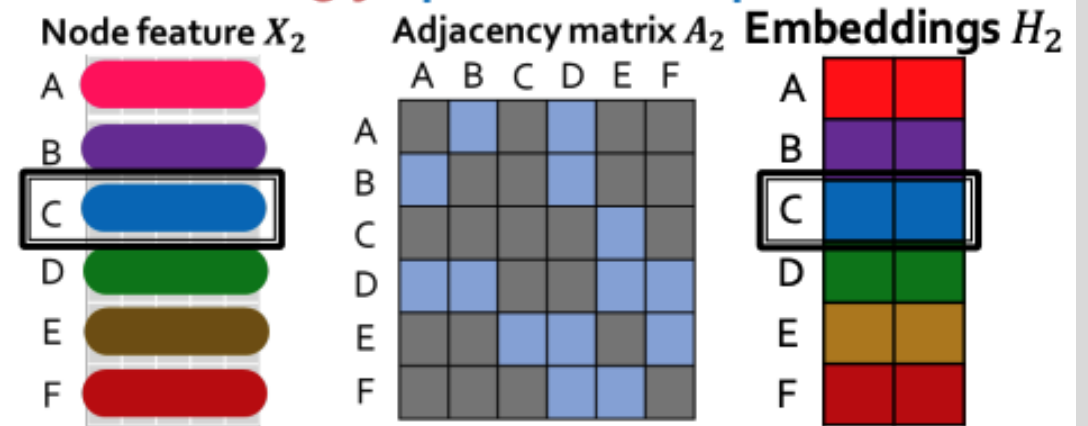


Embedding matrix of all nodes is permutation equivariant

1. The input node features and output embeddings have the same row indices.
2. We know the embedding computation of a given node with a GCN is permutation invariant.
3. So, permuting moves a given node to a new row in the feature matrix and the adjacency matrix, but also moves the output embedding to the same new row in the embedding matrix. **This is permutation equivariance.**



Permute the input, the output also permutes accordingly - permutation equivariant





Training a GNN

- **Supervised setting:** Minimize a loss function, $\mathcal{L}$, with respect to model parameters

$$\min_{\Theta} \mathcal{L}(y, f_{\Theta}(x))$$

- y is a node, edge, or graph target value
- $f_{\Theta}(x) = DEC[ENC((x))]$ is a composite function that includes the *DEC* (typically another NNet) which outputs the estimate, $\hat{y}$, of y given the *ENC* output which is a node, edge, or graph embedding from a GNN
- $\mathcal{L}$ is any appropriate loss function, e.g., L2 if y is a real number, or cross entropy if y is categorical
- **Unsupervised setting**
 - No labels available
 - Use the graph structure as self-supervision (e.g., link prediction)



Unsupervised GNN training

- One approach is to assume *similar* nodes should have similar embeddings using cross entropy loss:

$$\min_{\Theta} \mathcal{L} = - \sum_{u,v} \sum_{i=1}^C (y_i \log(\mathbf{z}_u \cdot \mathbf{z}_v))$$

where y_i is the actual class label based on a similarity function, S_i . That is, $y_i = 1$ if $S_i(u, v) > \gamma$ for some threshold γ .

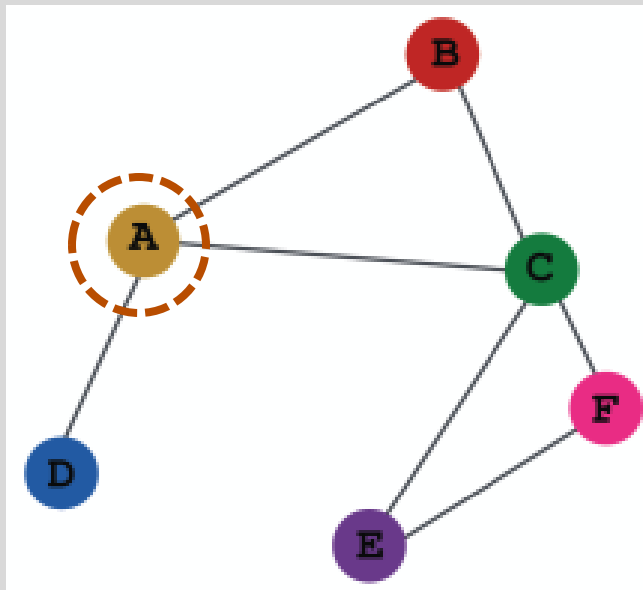
- Can apply *any node similarity function*, e.g., random walks



Supervised GNN training

- As with standard deep-learning, directly train the model for a supervised task such as node classification

Drug-drug
interaction network



Is the drug safe or
toxic?

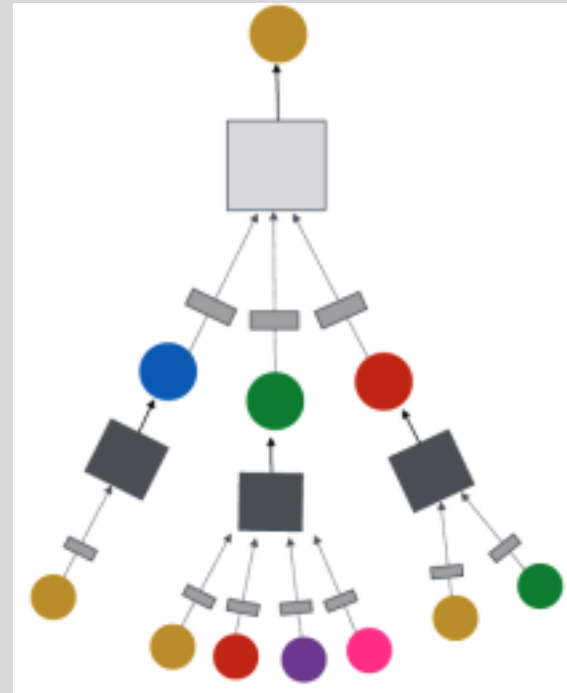


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



Supervised GNN training

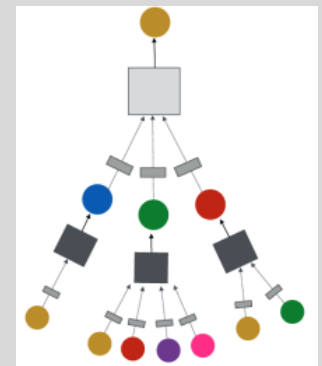
- Directly train the model for a supervised task such as node (or graph or edge) classification
- Apply cross-entropy loss

$$\mathcal{L} = - \sum_{v \in V} \boxed{y_v} \log[\sigma(\text{DEC}(\boxed{\mathbf{z}_v}))] + (1 - \boxed{y_v}) \log[1 - \sigma(\text{DEC}(\boxed{\mathbf{z}_v}))]$$

class label
encoder output (embedding)

- Loss gradient is computed for composite function $\text{DEC}(\text{ENC}(v))$ and learning parameters are updated for both
 - DEC can be just the identify function, i.e. just use $\sigma(\mathbf{z}_v)$

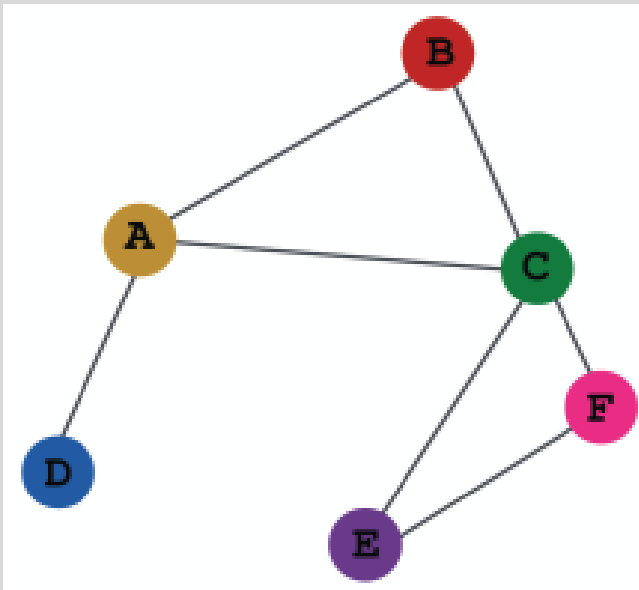
Safe or toxic?





GNN model design overview

(1) Define the neighborhood aggregation function



(2) Define a loss function

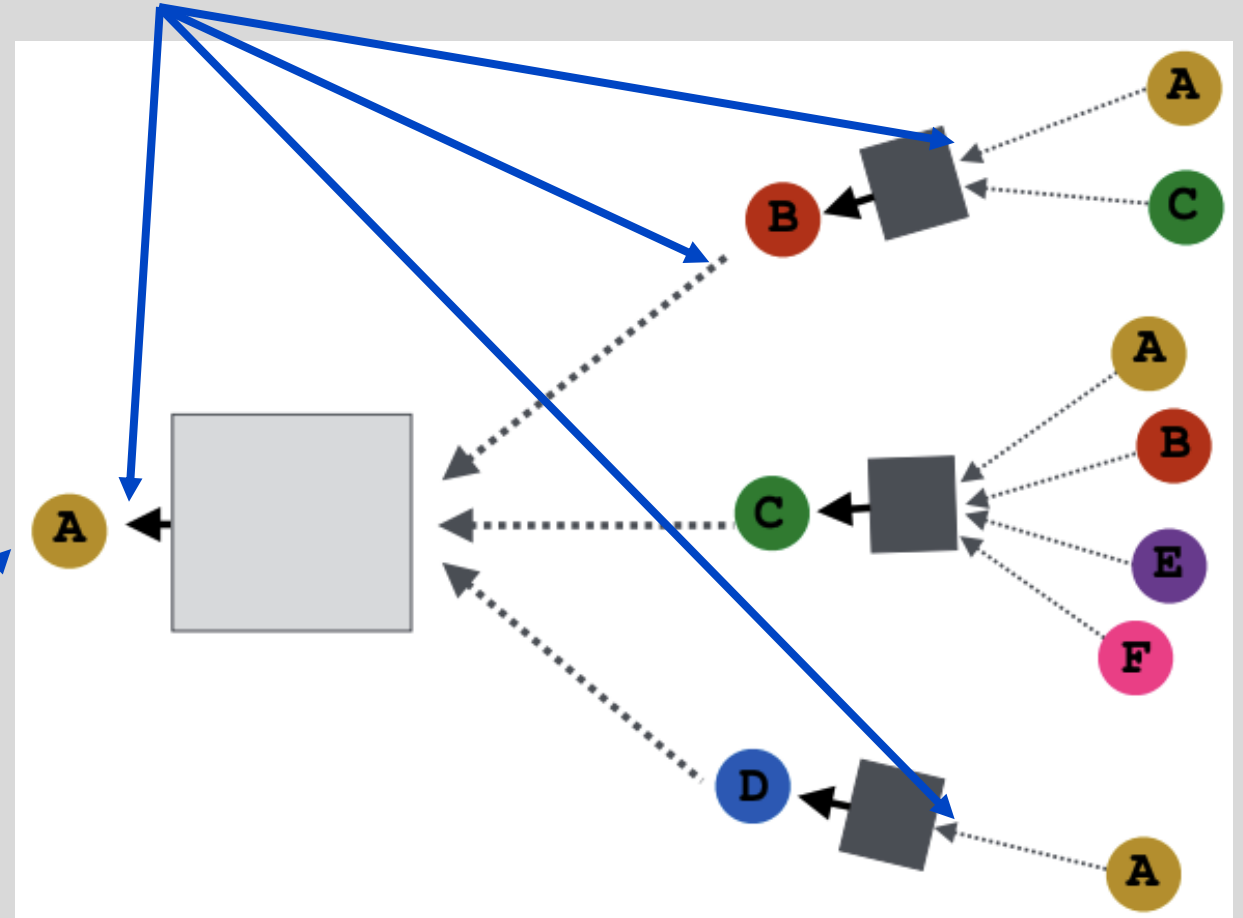
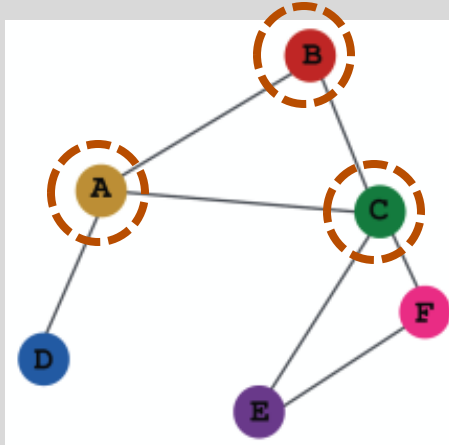


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



GNN model design overview



(3) Select training nodes and train the model with SGD

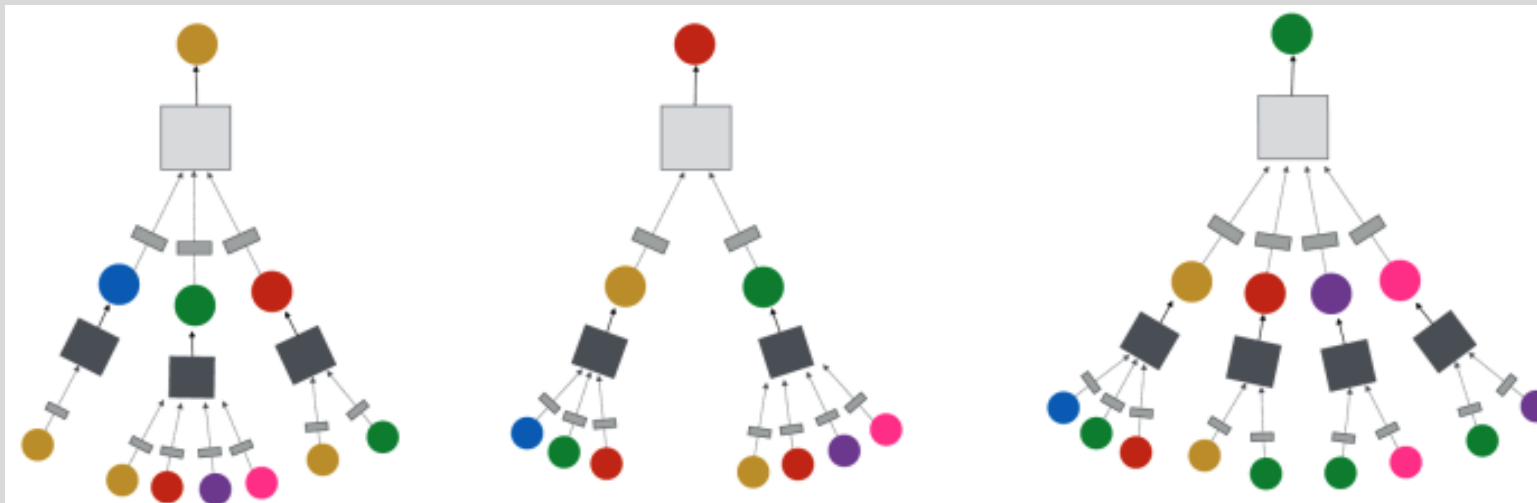


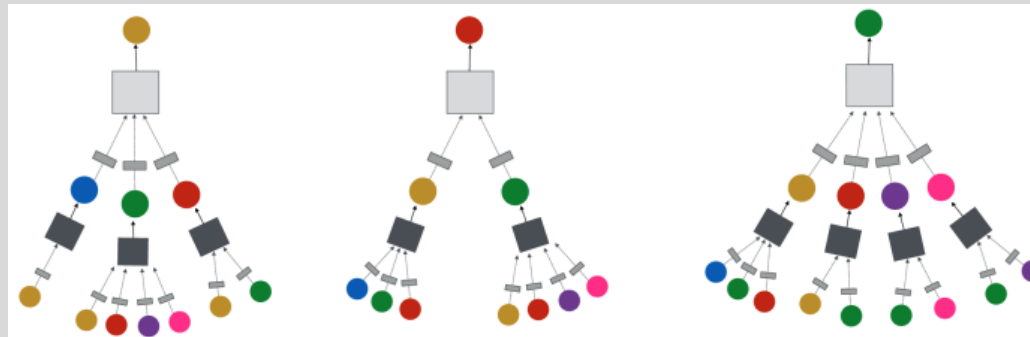
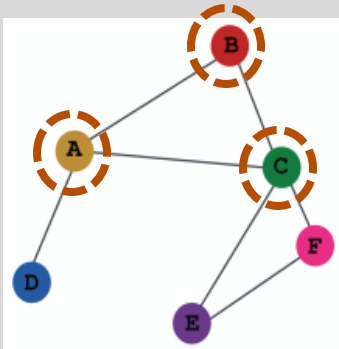
Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



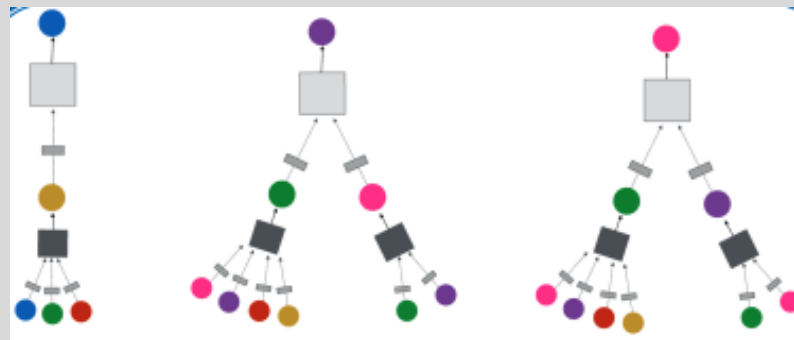
GCN embedding generation

Once the model is trained, we can generate embeddings for **any** nodes

Nodes seen in training



Nodes not seen in training



How is this possible?



GCN inductive capacity for unseen or new nodes

- The same aggregation learning parameters are shared for all nodes (or edges)
- Different parameters at each layer, i.e., $W_k \neq W_j$ for $k \neq j$
- What does this imply in the math?

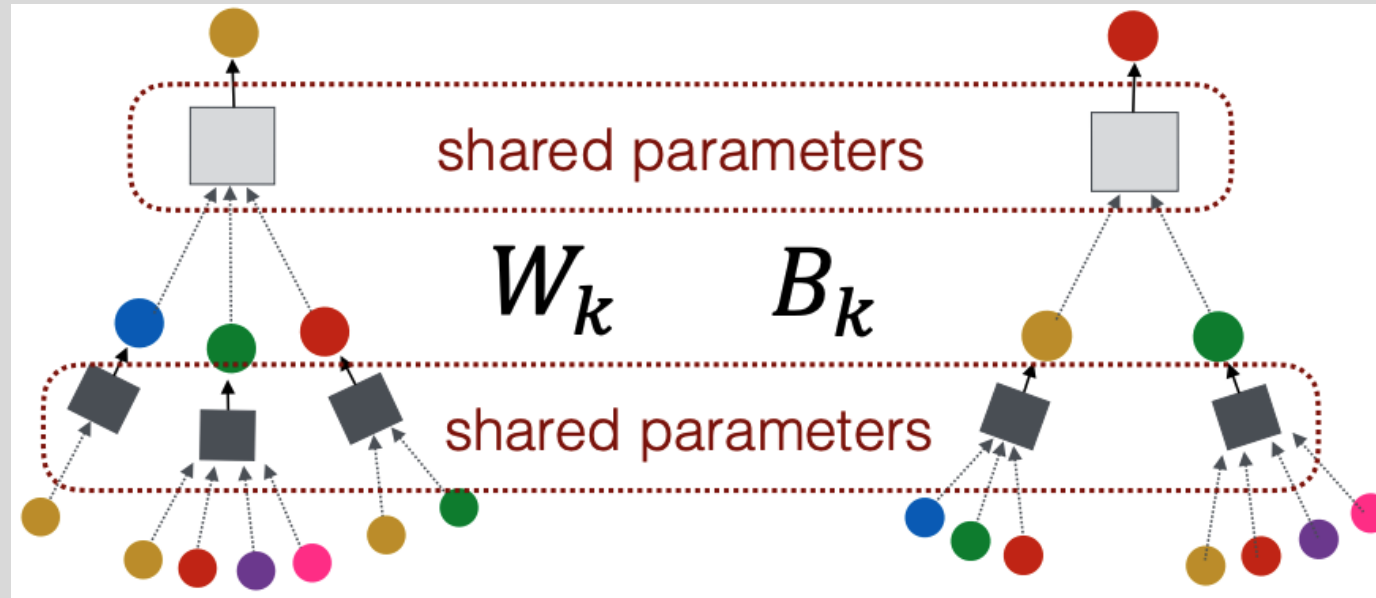


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



GCN inductive capacity for unseen or new nodes

Recall the math $\mathbf{H}^{(k+1)} = \sigma \left(\tilde{\mathbf{A}} \mathbf{H}^{(k)} \mathbf{W}^{(k)T} \right)$

- $\tilde{\mathbf{A}} \in \mathbb{R}^{|V| \times |V|}$
- $\mathbf{H}^{(k)} \in \mathbb{R}^{|V| \times d_k}$
- $\mathbf{W}^{(k)T} \in \mathbb{R}^{d_k \times d_{k+1}}$

where d_k and d_{k+1} are the latent embedding dimensions at layer k and $k + 1$, respectively

The operation $\tilde{\mathbf{A}} \mathbf{H}^{(k)} \in \mathbb{R}^{|V| \times d_k}$ and $\tilde{\mathbf{A}} \mathbf{H}^{(k)} \mathbf{W}_k^T \in \mathbb{R}^{|V| \times d_{k+1}}$

- The weight matrix is not dependent on the adjacency matrix dimensions
- We can create a new adjacency matrix to add or remove nodes (and edges) and process it with the same trained model to get embeddings

GCN inductive capacity for unseen or new nodes

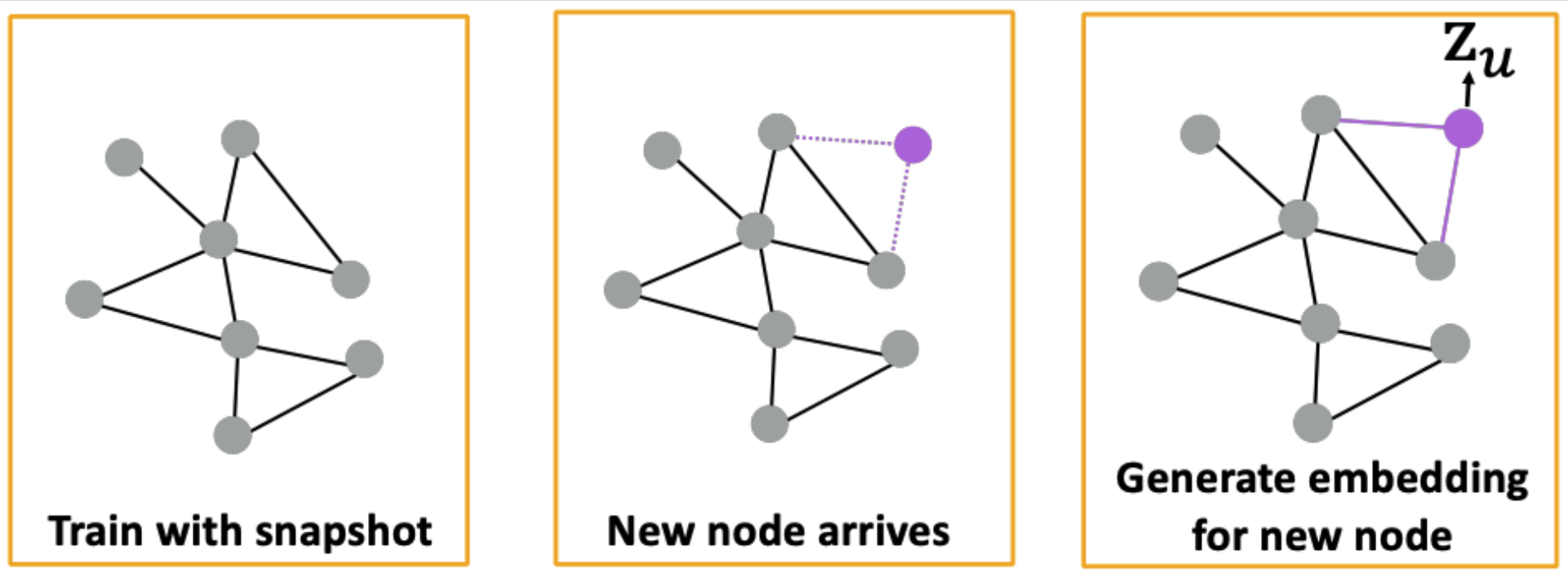


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>

- Many applications have dynamic graphs (social networks, device networks)
- Can generate new embeddings as needed and pass into downstream inference models
- Assumes new nodes have same features and inter-node similarity properties hold



GCN inductive capacity for new graphs

- Inductive node embeddings can generalize to an entirely unseen graph (it's just a new adjacency matrix)
- Example: train on known drug molecules and generate embeddings new molecules

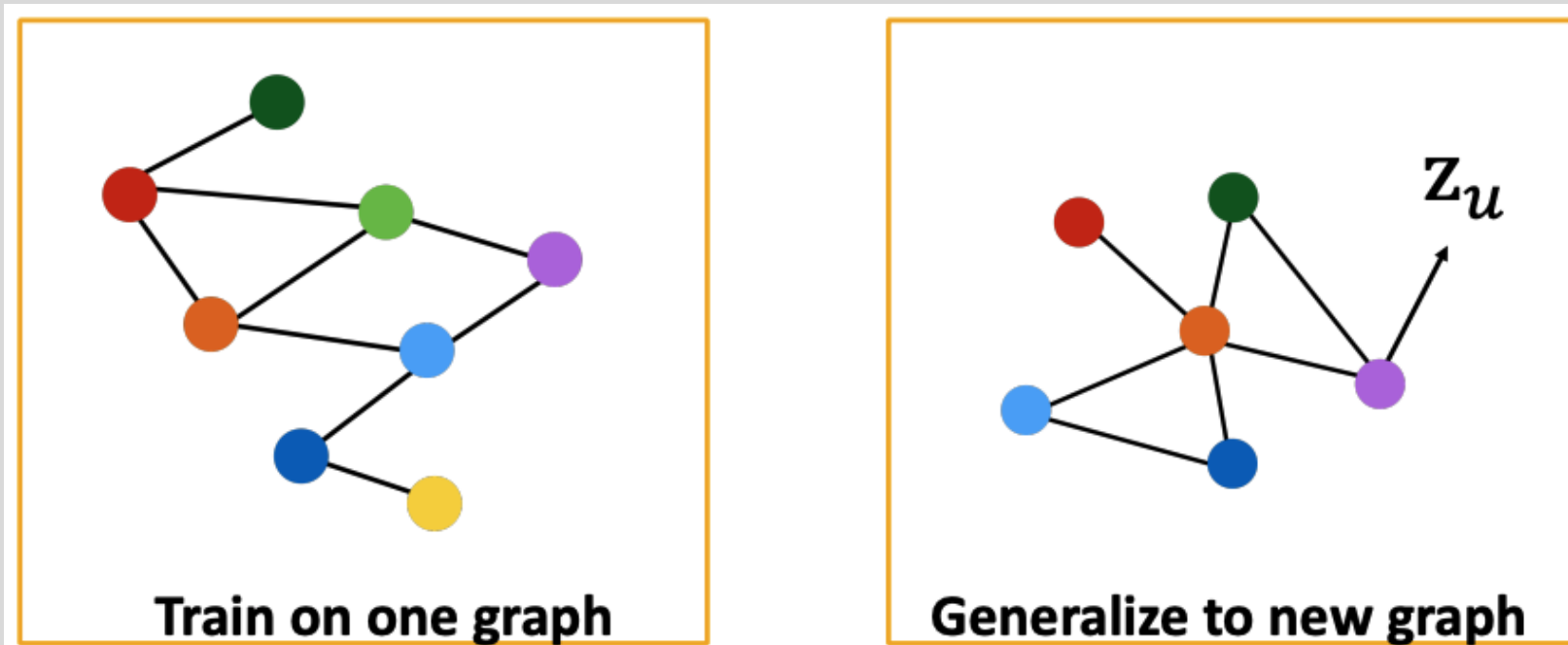


Image credit: Jure Leskovec, <http://cs224w.stanford.edu>



More on GNNs

Interactive visualizations

<https://visual-intelligence-umn.github.io/GNN-101/>

Model-Task

GCN - graph classification

GNN Model

Architecture

Input

GCNConv1

GCNConv2

GCNConv3

Global Pooling

Output

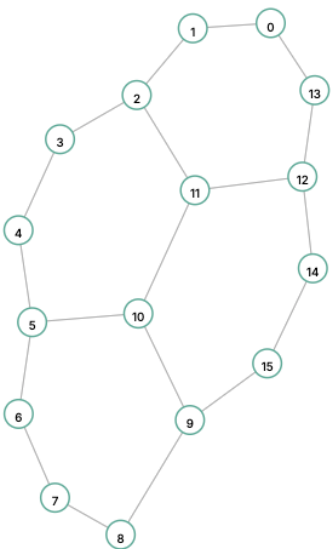
Input Graph

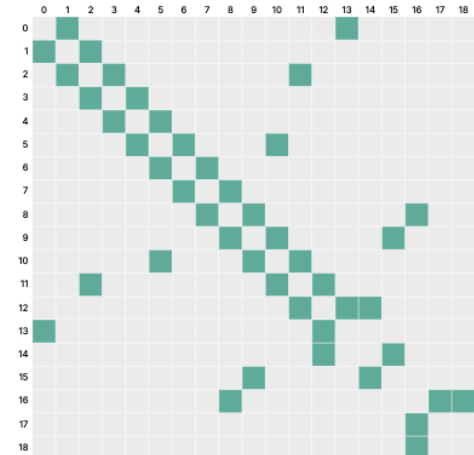
Each graph in the **MUTAG dataset** represents a chemical compound. Nodes are atoms and edges are bonds between atoms. The task is to predict whether a molecule is mutagenic on *Salmonella typhimurium* or not (i.e., can cause genetic mutations in this bacterium or not). The dataset has 188 graphs in total, with 150 graphs in the training set and 38 graphs in the test set.

Click to Predict!

Understanding Graphs and Adjacency Matrices

A graph can be visualized as either a node-link diagram or an adjacency matrix. Hover over the nodes, edges, matrix cells, or matrix labels to highlight their connections.





CPSC 8810 Machine Learning for Graphs

59

